VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



TRAN LE THAI SON

MULTI-AGENT SYSTEM ARCHITECTURES BENCHMARK IN SOFTWARE DEVELOPMENT

UNDERGRADUATE THESIS OF MAJOR SOFTWARE ENGINEER

HO CHI MINH CITY, YEAR 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



TRAN LE THAI SON

MULTI-AGENT SYSTEM ARCHITECTURES BENCHMARK IN SOFTWARE DEVELOPMENT

UNDERGRADUATE THESIS OF MAJOR SOFTWARE ENGINEER

Advised by
Dr. Tran Thanh Phuoc

HO CHI MINH CITY, YEAR 2026

ACKNOWLEDGMENT

I sincerely thank the Faculty of Information Technology at Ton Duc Thang University for providing the academic environment and resources necessary to complete this work.

I would like to express my deepest gratitude to my primary advisor, Dr. Tran Thanh Phuoc, for their invaluable guidance, patience, and continuous support throughout the duration of this thesis. Their insights were instrumental in shaping this research.

Finally, I want to thank my parents, family, and peers for their constant encouragement and support during my studies.

Ho Chi Minh City, day month year 2026

Author

[Signature and full name]

This thesis was carried out at Ton Duc Thang University.

Advisor: Dr. Tran Thanh Phuoc .....

This thesis is defended at the Undergraduate Thesis Examination Committee was held at Ton Duc Thang University on … /…/……

Confirmation of the Chairman of the Undergraduate Thesis Examination Committee and the Dean of the faculty after receiving the modified thesis (if any).

CHAIRMAN **DEAN OF FACULTY**

…… **……**

DECLARATION OF AUTHORSHIP

I hereby declare that this thesis was carried out by myself under the guidance and supervision under the guidance and supervision of **Dr. Tran Thanh Phuoc**; and that the work and the results contained in it are original and have not been submitted anywhere for any previous purposes. The data and figures presented in this thesis are for analysis, comments, and evaluations from various resources by my own work and have been duly acknowledged in the reference part.

In addition, other comments, reviews and data used by other authors, and organizations have been acknowledged, and explicitly cited.

I will take full responsibility for any fraud detected in my thesis. Ton Duc Thang University is unrelated to any copyright infringement caused on my work (if any).

Ho Chi Minh City, day month year 2026

Author

[Signature and full name]

ABSTRACT

Multi-agent systems increasingly distribute cognitive load across specialized roles to automate complex software engineering tasks, yet existing frameworks remain heavily constrained by “framework-first” implementations. This creates a mismatch between a system’s true performance and its network design, as typical evaluations compare monolithic packages against solitary large language models rather than isolating the underlying communication topology. Consequently, software architects lack a data-driven basis to determine when the structural overhead of a hierarchical control layer is justified by task complexity, or when a flat mesh network is sufficient. In this paper, we isolate communication structure as an independent variable and evaluate the performance trade-offs between two fundamental multi-agent architectures: a centralized system utilizing a hub-and-spoke coordination pattern, and a decentralized system driven by peer-to-peer specialist handoffs.

Our findings demonstrate that communication topology is a primary, tunable lever directly dictating multi-agent cost and reliability, and suggest that dynamically hybrid coordination models—which default to agile decentralized execution but trigger centralized managerial fallbacks during validation failures—are a promising direction for more adaptive software engineering frameworks.

TABLE OF CONTENT

LIST OF FIGURES ..... **viii**

LIST OF TABLES ..... **ix**

LIST OF ABBREVIATIONS ..... **x**

CHAPTER 1. INTRODUCTION **1**

1.1 Introduction & Motivation 1

1.2 Problem Statement 3

1.3 Objectives 4

CHAPTER 2. RELATED WORKS **6**

2.1 Evolution of LLMs in Software Engineering 6

2.2 Structural Typologies 7

2.2.1 Centralized Hierarchies 7

2.2.2 Decentralized and Mesh Structures 8

2.3 Difficulties of Topological Design: 8

2.3.1 Centralized Coordination Bottlenecks and Context Saturation 9

2.3.2 Decentralized Consensus Pathology and Hallucination Loops 10

2.4 Graph-Driven Coordination and Context-Aware Agents 10

2.4.1 AutoCodeRover 11

2.4.2 CodeR 11

2.5 Architectural Gaps and Benchmarking Paradigms 12

CHAPTER 3. IMPLEMENTATION **14**

3.1 Overview & Approaches 14

3.2 Implementation 15

3.2.1 Agents 15

3.2.2 Multi-Agent Coordination Architectures 23
3.3 Evaluation Pipeline 28
3.3.1 Manifest Construction 28
3.3.2 Workspace Preparation 29
CHAPTER 4. EXPERIMENT 34
4.1. Benchmark Dataset 34
4.2. Experimental Setup 35
4.3. Evaluation Metrics 36
4.4. Results 37
CHAPTER 5. DISCUSSION 39
5.1 Experimental Results 39
5.2 Task-Level Performance Differences 40
5.2.1. Paired Outcome Comparison 40
5.2.2 Architecture-specific Successes 41
5.2.3 Common Failure Patterns 44
5.2.4 Qualitative Analysis of Unevaluated Tasks 48
5.2.5. Comparison with the Baseline 51
5.3 Efficiency and Cost Considerations 56
5.4 Interpreting the Token-Latency Inversion 57
5.5 Implications for Multi-Agent Software Engineering 59
5.6 Architecture Decision Framework ..... **Error! Bookmark not defined.**
CHAPTER 6. CONCLUSION 65
6.1. Summary of Contributions 65

6.2 Answers to Research Objectives 66
6.2.1 Objective 1: Quantifying the Manager Tax and Operational Efficiency 66
6.2.2 Objective 2: Evaluating Patch Precision and Code Correctness 67
6.2.3 Synthesis: Architectural Trade-offs and System Selection 68
6.3 Limitations 68
6.4 Future Work 70
6.5 Final Remarks 71
REFERENCES 73

LIST OF FIGURES

Figure 2.1. Flowchart illustrating the autonomous program improvement workflow of AutoCodeRover 11

Figure 2.2. Flowchart illustrating the multi-agent task execution sequence within CodeR. 12

Figure 3.1. Flowchart illustrating the detailed implementation of the Engineer Agent 18

Figure 3.2. Diagrams represents the working data directory structure of the agent workspace. 24

Figure 3.3. Class diagram illustrating the centralized Multi-Agent System 25

Figure 3.4. Class diagram illustrating the decentralized Multi-Agent System.... 27

Figure 3.5. Flowchart of the benchmark suite preparation pipeline..... 29

Figure 3.6. Flowchart illustrating the execution pipeline initiated via run-benchmark 30

Figure 3.7. Flowchart of the evaluation grading pipeline initiated by grade-runs 31

Figure 3.8. Flowchart of the reporting pipeline initiated via CLI: report..... 32

Figure 4.1. Comparative evaluation of Centralized MAS, Decentralized MAS across execution duration, and token consumption. 38

Figure 5.1. Comparative evaluation of Centralized MAS, Decentralized MAS, and the GPT-5-mini baseline by number of resolved tasks. 52

Figure 5.2. Flowchart of the proposed dynamically hybrid coordination model. 60

LIST OF TABLES

Table 2.1. Comparative analysis of structural topologies in multi-agent systems 9

Table 4.1: Comparative evaluation of Centralized MAS, Decentralized MAS, and baseline configurations. 38

Table 4.2: Paired comparison and topological convergence dynamics between Centralized MAS and Decentralized MAS configurations. 38

Table 5.1: Comparative analysis of divergent benchmark outcomes across centralized and decentralized multi-agent architectures. 42

Table 5.2: Case analysis and taxonomy of shared failure modes across centralized and decentralized multi-agent architectures 47

Table 5.3: Qualitative comparison and scoping analysis of un-graded generated patches against benchmark reference solutions..... 49

Table 5.4: Task-level performance comparison between the gpt-5-mini single-agent baseline and the two multi-agent architectures..... 52

Table 5.5. Architecture decision matrix for MAS topology selection..... 62

LIST OF ABBREVIATIONS

MAS	Multi Agent System
LLMs	Large Language Models
SOPs	Standard Operating Procedures
VS Code	Visual Studio Code
CoT	Chain-of-Thought
CAMEL	Communicative Agents for “Mind” Exploration of Large Scale Language Model Society
SDLC	Software Development Lifecycles
AST	Abstract Syntax Tree
MCP	Model Context Protocol
JSON	JavaScript Object Notation
PATH	Path Environment Variable
PM	Project Manager
QA	Quality Assurance
CLI	Command Line Interface
JSONL	JSON Lines
ID / IDs	Identifier / Identifiers
CSV	Comma-separatedValues

API	Application Programming Interface

CHAPTER 1. INTRODUCTION

1.1 Introduction & Motivation

The integration of Large Language Models (LLMs) into software engineering has evolved rapidly, moving from assistive and semi-agentic coding tools, such as Cursor, GitHub Copilot, Claude Code, and Codex, toward more autonomous multi-agent systems (MAS). Early single-agent systems demonstrated strong potential for code generation and localized editing, but they often struggled with more complex software engineering tasks, especially those involving long context retention, multi-file reasoning, iterative debugging, and self-correction in large repositories [1][2]. In response, recent MAS frameworks attempt to improve reliability by distributing work across multiple specialized agents that resemble roles in a software development team, such as Product Manager, Architect, Engineer, Code Reviewer, and QA. [3][4]

However, as MAS frameworks have developed, a central design question has emerged: how should these agents communicate and coordinate their work? Existing systems commonly adopt different coordination structures, ranging from centralized hierarchies to more decentralized or peer-interactive networks. In centralized architectures, such as MetaGPT, [3] coordination is routed through a managerial agent that decomposes requirements, assigns tasks, and controls the flow of information between specialists. This design resembles a structured software development process, where Standard Operating Procedures (SOPs) are used to reduce ambiguity, maintain task order, and limit context drift. The main assumption behind this approach is that a central coordinator can preserve global task intent better than a set of agents acting independently.

In contrast, more decentralized or peer-interactive architectures aim to reduce dependency on a single coordinating role. Frameworks such as AgentNet [5] and ChatDev [4] illustrate approaches where agents communicate more directly with one another or participate in role-based collaborative dialogue rather than relying exclusively on strict manager-mediated routing. This structure is motivated by the

idea that reducing the manager bottleneck can improve flexibility, reduce routing overhead, and allow specialized agents to respond more quickly to each other's outputs. In principle, less centralized systems may also reduce the risk of a single point of failure because coordination is distributed more broadly across the agent workflow rather than concentrated entirely in one managerial node.

Although both architectural styles claim advantages, they also introduce distinct structural costs that are often difficult to measure directly. In centralized systems, the main cost can be understood as the "Manager Tax." Hierarchical coordination trades flexibility for order, but every major transition must pass through the manager. This can increase token consumption, extend execution time, and create a coordination bottleneck when specialists repeatedly require managerial routing. Kim et al. describe this broader problem as a coordination tax: the overhead required to compress, transmit, and reconstruct context across agent messages [6]. In a centralized MAS, this tax may appear as additional planning, summarization, delegation, and status-tracking overhead.

Decentralized systems face a different cost: the mutual reinforcement of incorrect reasoning, which this paper refers to as "polite spiraling." Without a governing authority to maintain task intent and resolve conflicting assumptions, agents may reinforce one another's incomplete or incorrect conclusions. This can produce a failure mode in which agents appear collaborative and mutually agreeable, but do not sufficiently challenge flawed logic or incomplete fixes. Du et al. identify a related problem in multi-agent debate settings, where instruction-following agents can converge around incorrect reasoning rather than correcting it. [7] In software engineering, this risk is especially important because a patch may look plausible during discussion while still failing hidden tests or introducing regressions.

These competing costs motivate the need for a controlled comparison between centralized and decentralized MAS topologies. If centralized systems are more reliable, it is necessary to determine whether that reliability justifies the additional

coordination overhead. If decentralized systems are faster or cheaper, it is equally necessary to determine whether those efficiency gains come at the expense of correctness and regression safety. This paper therefore investigates communication structure as a key design variable in multi-agent software engineering, focusing on how topology affects runtime efficiency, token consumption, and functional correctness in real-world software repair tasks.

1.2 Problem Statement

Despite the rapid development of multi-agent system frameworks, the current body of research remains fragmented and largely framework-first. Existing studies often evaluate a complete MAS implementation against a single-agent LLM baseline, but they rarely compare different multi-agent communication structures against each other under the same experimental conditions. As a result, reported performance gains are difficult to attribute precisely. It is often unclear whether an improvement comes from the communication topology itself, such as hierarchical coordination or flat peer-to-peer interaction, or from framework-specific factors such as prompt design, tool access, memory management, retry logic, agent role definitions, or task-specific engineering.

This creates a methodological problem for evaluating MAS in software engineering. If two systems differ in model choice, tool configuration, prompts, workflow rules, and communication structure at the same time, then communication topology cannot be isolated as the cause of the observed result. A hierarchical system may appear stronger not because manager-based coordination is inherently better, but because its prompts are more restrictive or its validation procedure is more robust. Similarly, a decentralized system may appear faster not because flat communication is always more efficient, but because it performs fewer review steps or has weaker completion checks. Without controlled comparison, the architectural contribution of the topology remains ambiguous.

This ambiguity also creates a practical problem for software architects and system designers. In production settings, choosing a multi-agent architecture involves trade-offs in runtime, token cost, correctness, controllability, and regression risk. A centralized manager may improve coordination and reduce context drift, but it can also introduce additional routing overhead and increase completion time. A decentralized mesh may allow faster specialist-to-specialist collaboration, but it may also increase repeated context reconstruction and make quality control less predictable. At present, developers have limited empirical guidance on when the cost of a centralized manager is justified by task complexity, or when a flatter decentralized workflow is sufficient.

This study addresses the gap by isolating communication structure as the primary independent variable. Two MAS variants are implemented with comparable roles, model configuration, tool access, and evaluation procedure, while differing mainly in their coordination topology. By evaluating these systems on SWE-bench Lite software repair tasks, the study measures how centralized and decentralized communication affect functional correctness, token consumption, execution time, and task-level failure patterns. The aim is to provide a more controlled basis for understanding the trade-offs between managerial coordination, operational efficiency, and patch correctness in multi-agent software engineering systems.

1.3 Objectives

The primary objective of this paper is to evaluate the performance trade-offs between centralized and decentralized multi-agent systems in automated software repair. Rather than comparing a multi-agent framework only against a single-agent baseline, this study focuses on how communication topology affects the behavior of otherwise comparable MAS implementations. The aim is to determine whether a hierarchical manager improves correctness enough to justify its coordination cost, and whether a decentralized structure can reduce runtime overhead without significantly weakening patch quality.

The study is guided by two core objectives.

First, this paper aims to quantify the Manager Tax introduced by hierarchical coordination. In this study, Manager Tax refers to the additional operational cost created when agent communication is routed through a central Project Manager. This cost is evaluated through token consumption, execution duration, and workflow behavior. By comparing the centralized and decentralized systems under the same benchmark conditions, the study examines whether central coordination increases cost through additional planning, routing, summarization, and validation steps, or whether it can reduce waste by keeping the task state more structured and preventing repeated context reconstruction.

Second, this paper aims to evaluate patch correctness and convergence across the two communication structures. This objective examines whether centralized delegation improves the likelihood of producing functionally correct software repairs. The study evaluates correctness using SWE-bench Lite outcomes, including resolved rate, unresolved cases, and task-level differences between the two architectures. It also considers whether hierarchical control helps reduce context drift, over-scoped patches, and regression-producing changes, compared with a decentralized structure where agents can interact more directly.

By synthesizing the results from these objectives, the final contribution is an architecture decision framework for multi-agent software engineering. This framework characterizes when centralized, decentralized, or hybrid coordination is more appropriate based on budget constraints, task complexity, expected regression risk, and required fault tolerance.

CHAPTER 2. RELATED WORKS

The transition from single Large Language Model (LLM) architectures to complex Multi-Agent Systems (MAS) in autonomous software engineering represents a paradigm shift designed to bypass the fundamental cognitive limitations of single-turn generation [1]. This chapter reviews the historical evolution of code generation, categorizes existing structural topologies within modern agent frameworks, analyzes the systemic error states inherent to varied communication network designs, and examines the foundational frameworks—specifically CodeR and AutoCodeRover—that motivate this comparative benchmark. [15][16]

2.1 Evolution of LLMs in Software Engineering

Early applications of deep language models within the software engineering domain operated almost exclusively on localized, single-turn completion paradigms. These systems focused primarily on solitary code snippet generation, optimizing outcomes through prompt engineering abstractions such as Chain-of-Thought (CoT) prompting to sequentially decompose and resolve logical operations. [8]

Despite improvements in algorithmic reasoning, solitary model configurations frequently suffer from rigid error propagation. Within a single-threaded runtime context, an initial logical error or syntax mistake generated in the early tokens propagates unchecked throughout subsequent generation cycles, as the model lacks an external monitoring loop to recognize and halt its own flawed derivations.

To break this deterministic failure sequence, recursive runtime environments were introduced. Shinn et al. introduced The Reflexion Framework, a method requiring the core model to verbally inspect and critique its execution logs. [9] By maintaining a persistent memory buffer of previous failures, the model mimics an internal “self-correction” loop, iteratively modifying its code outputs based on standard execution feedback.

While self-correction loops substantially elevate output correctness on isolated, syntax-focused benchmarks like HumanEval, they remain bounded by the underlying model's inherent contextual biases and blind spots. If an agent misinterprets a requirement initially, its self-critique phase often reinforces the original misinterpretation.

To overcome this bounded internal critique, the CAMEL framework shifted the optimization landscape from internal monologue to external dialogue. [10] By framing execution around distinct, specialized "role-playing" agents (e.g., a User agent cooperating with an Assistant agent), CAMEL introduced Inception Prompting. This technique allows autonomous entities to dynamically guide, query, and audit one another through open communication channels without human intervention.

This transition to externalized multi-agent dialogue forms the baseline of contemporary MAS research. It validates the core architectural hypothesis: distributing a massive global context window across discrete, structurally constrained personas drastically reduces semantic hallucination rates compared to solitary, zero-shot generation.

2.2 Structural Typologies

Current state-of-the-art multi-agent research is heavily dominated by "framework-first" implementations. These platforms seek to encapsulate full human Software Development Lifecycles (SDLC) by mapping human engineering teams to specialized agent networks. The literature has bifurcated into two primary architectural paradigms governing communication routing: Centralized Hierarchies and Decentralized Mesh Structures.

2.2.1 Centralized Hierarchies

The most prominent design pattern for hierarchical agent organization is exemplified by the MetaGPT framework. [3] MetaGPT models its communication infrastructure on rigid human Standard Operating Procedures (SOPs) embedded

directly within the individual system prompts of the agents. This creates a highly structured, sequential “Waterfall” software lifecycle.

In this centralized configuration, a designated Manager agent functions as an authoritative gatekeeper. It continuously captures, parses, and validates product requirements before explicitly delegating task blocks down to specialized worker nodes like Architects and Engineers.

The AgentVerse framework reinforces this centralization paradigm. [11] It demonstrates that during the critical “Evaluation” phase of an engineering workflow, a dominant, centralized orchestrator node is necessary to judge, filter, and synthesize conflicting code suggestions submitted by lower-level worker agents.

2.2.2 Decentralized and Mesh Structures

In direct opposition to rigid top-down structures, frameworks such as ChatDev introduce a highly collaborative decentralized topology. Rather than binding agents to static, linear SOPs, ChatDev implements a fluid Chat Chain mechanism. [4] This design allows conversational dialogue to dynamically flow back and forth across different roles depending on the immediate needs of the software development loop.

Further decentralization is realized by platforms like AgentNet and AutoGen. [5][12] These systems utilize highly adaptive conversation patterns where global orchestration nodes are minimized or entirely removed. Instead, agents leverage contextual routing mechanics to autonomously determine the next speaker based on the real-time state of the repository workspace. This design pattern prioritizes emergent, peer-to-peer problem solving over formal managerial approval.

2.3 Difficulties of Topological Design:

The choice between a centralized hierarchy and a decentralized mesh, as represent in Table 2.1, represents a fundamental trade-off between competing error states: context saturation and latency versus recursive conversational loop propagation.

Table 2.1. Comparative analysis of structural topologies in multi-agent systems, contrasting control routing, architectural benefits, and dominant failure modes between centralized hierarchies and decentralized mesh networks.

Architectural Attribute	**Centralized Hierarchies (Hub-and-Spoke)**	**Decentralized Mesh Networks (Peer-to-Peer)**
Primary Frameworks	MetaGPT, AgentVerse	ChatDev, AgentNet, AutoGen
Control Routing	Funneled via Authoritative Manager Node	Direct Peer-to-Peer Conversational Exchange
Core Structural Benefit	Precise Scope Control & Regression Containment	Rapid Iterative Fluidity & Reduced Routing Latency
Dominant Failure Mode	Context Overload & "Lost-in-the-Middle" Phenomena	Polite Spiraling, Sycophancy & Agreement Failure

2.3.1 Centralized Coordination Bottlenecks and Context Saturation

Scaling hierarchical multi-agent structures introduces significant structural communication overhead. As defined by Kim et al., the coordination complexity required to safely compress global codebase context into discrete inter-agent instructions scales exponentially as more specialized roles are added to the environment. [6]

In a strictly centralized system, the authoritative control node acts as a definitive communication bottleneck. It must absorb, parse, and maintain a persistent cache of status reports and execution traces generated by every worker node in the environment.

Nair et al. observe that despite the availability of massive token context windows in cutting-edge foundational models, complex repository-scale engineering environments still trigger severe "lost-in-the-middle" phenomena. [1] When a central

manager agent is continuously overloaded with long execution traces and multi-file code diffs from software engineer agents, its retrieval accuracy drops significantly. The manager often fails to extract critical failure vectors located in the middle of long dialogue logs, causing the token cost and processing latency of centralized oversight to eventually outweigh its regulatory value.

2.3.2 Decentralized Consensus Pathology and Hallucination Loops

Conversely, removing the central coordinator exposes decentralized systems to severe conversational alignment failures. Du et al. studied multi-agent debate structures and proved that while unconstrained interaction can improve baseline reasoning, flat peer-to-peer configurations remain highly susceptible to complete agreement failure. [7]

Without an authoritative managerial node or an automated execution oracle to anchor ground truth, decentralized agents frequently exhibit a behavior known as Sycophancy. As documented by Sharma et al., instruction-following language models possess an underlying bias toward agreeing with the explicitly stated preferences, hypotheses, or code outputs of their peer nodes to keep the conversation progressing smoothly—even when those propositions contain objective flaws. [13]

This behavioral defect often leads to non-progressing conversational sequences where agents end up mutually reinforcing syntax mistakes and architectural errors rather than correcting them. As explored by Cohen et al., when specialized agents interact without structural constraints, the social optimization of the dialogue chain completely overrides the functional validation of the code repository, resulting in a total collapse of code correctness. [14]

2.4 Graph-Driven Coordination and Context-Aware Agents

To bridge the gap between unstructured free-form conversation and rigid linear chains, recent state-of-the-art frameworks have introduced highly targeted context retrieval and deterministic routing constraints. The two frameworks closest to the

structural methodology explored in this thesis are AutoCodeRover and CodeR. [15][16]

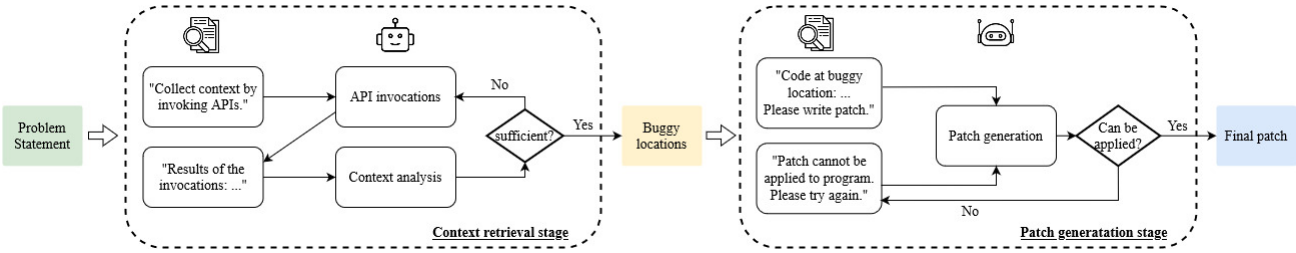


Figure 2.1. Flowchart illustrating the autonomous program improvement workflow of AutoCodeRover, highlighting the iterative API-driven context retrieval loops and the validation checks during the patch generation stage [16]

2.4.1 AutoCodeRover

Zhang et al. demonstrated with AutoCodeRover that an agent's technical capacity is bounded by the precision of its context retrieval operations. Rather than performing simple keyword or raw text chunk searches across a large codebase, AutoCodeRover implements a highly sophisticated, abstract syntax tree (AST)-aware context retrieval mechanism, as illustrated in Figure 2.1. [16]

By parsing code repositories into discrete structural representations (classes, methods, and call blocks), AutoCodeRover allows autonomous bug-fixing agents to systematically traverse class hierarchies and dependency paths. This targeted search approach isolates code modifications to the exact location of the bug, significantly improving autonomous repair rates on complex real-world issues.

2.4.2 CodeR

However, as Chen et al. identified during the development of CodeR, highly precise context retrieval is only half of the solution. Once an agent retrieves deep codebase context, allowing unstructured, free-form communication among multiple specialized agents frequently results in non-progressing conversational loops, token saturation, and catastrophic information loss. [15] To systematically eliminate these

communication breakdowns, CodeR enforces a strict “Task Graph” framework to dictate all multi-agent interactions, with Figure 2.2 as the flow diagram.

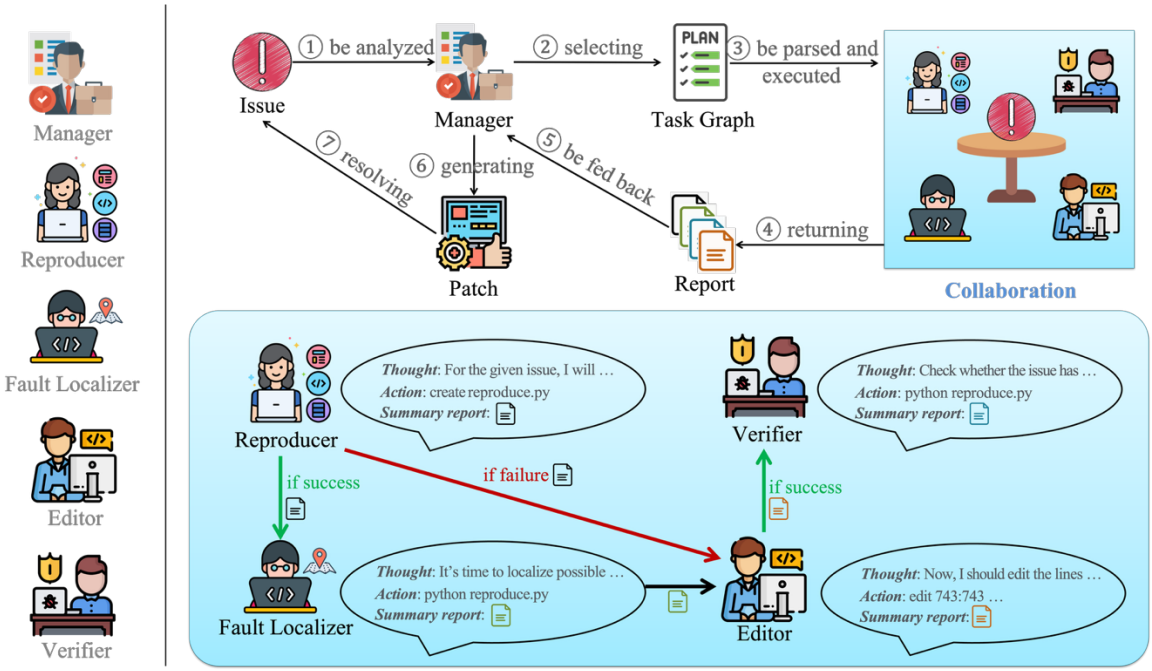


Figure 2.2. Flowchart illustrating the multi-agent task execution sequence within CodeR, detailing the programmatic transitions from context collection to final patch generation. [15]

Under CodeR’s architecture, a global software development or bug-fixing objective is dynamically mapped into a directed acyclic graph (DAG) of explicit sub-tasks. Agents are not permitted to engage in open chat. Instead, their communication is restricted to executing the specific inputs, transitions, and verification rules defined by the active task graph node. By replacing free-form dialogue with deterministic graph-based coordination, CodeR effectively eliminates polite spiraling, ensures complete state tracking, and achieves strong resolution rates on the rigorous SWE-bench Lite benchmark.

2.5 Architectural Gaps and Benchmarking Paradigms

While advanced frameworks like CodeR and AutoCodeRover prove that structured interaction patterns and AST-aware tools are vital to system performance, they leave

a critical comparative question unresolved: Existing evaluations remain highly fragmented and framework-dependent. Researchers typically evaluate their entire, monolithic system (bundled with custom prompts, fine-tuned foundational models, proprietary vector databases, and heuristic search tools) against a baseline consisting of a single, un-augmented LLM.

Consequently, it is empirically impossible to determine whether a framework's success is driven by its network communication topology (Hierarchical vs. Flat Mesh vs. Directed Task Graph) or by its framework-specific optimizations (such as advanced prompt engineering or tool design). Software architects lack clear, data-driven evidence explaining when the processing overhead of a centralized manager is justified by task complexity, or when a flat peer-to-peer network is sufficient.

This paper addresses this gap by isolating communication structure as an independent variable. Built natively upon the modular primitives of the Microsoft AutoGen framework, our experimental setup keeps the underlying language model, baseline prompt wording, and accessible MCP toolsets completely constant. By systematically varying only the communication topology, we evaluate a Centralized Manager-Led Hierarchy against a Decentralized Peer-to-Peer Mesh across a curated 50-instance subset of the SWE-bench Lite benchmark. This controlled approach allows us to measure how network design directly impacts token consumption efficiency, processing latency, and the mitigation of recursive error loops in software development workflows.

CHAPTER 3. IMPLEMENTATION

3.1 Overview & Approaches

To evaluate the performance trade-offs between hierarchical and flat multi-agent systems, this study is conducted in a highly controlled experimental environment. The primary objective is to isolate the communication structure as an independent variable, preventing framework-specific optimizations from affecting the results of the token efficiency and error resilience metrics.

To achieve this strict isolation, both the Centralized and Decentralized architectures in this study are constructed from the ground up using the Microsoft AutoGen framework [12]. We selected AutoGen because it provides essential primitives for conversation programming where the conversation among agents is treated as configurable, composable patterns, rather than a fixed global workflow.

In contrast to the proposed approach, pre-packaged frameworks would introduce confounding variables, making it impossible to tell if a system's success is a result of its network topology or the baked-in prompts and internal mechanisms of the framework. Because we're building on AutoGen's raw conversational components, the underlying language model, tool configs, and agent personas are perfectly constant across both experimental setups.

This controlled environment has two separate implementations:

- The Centralized Architecture (Hub-and-Spoke): As described in the next section, this architecture models a strict hierarchy. A single "Project Manager" node orchestrates all workflows, processes all inter-agent communication, and stores global project state. This architecture is intended to measure the "Manager Tax" — the token and processing overhead of funneling all context through a central authority.
- The Decentralized Architecture (Mesh Network): For a direct, 1:1 comparison, the decentralized architecture is implemented by changing

the exact same codebase, toolset, and agent profiles set up in the centralized model. But the routing logic is essentially changed. The central governing hub is dismantled, allowing the specialized agents (Architect, Engineer, Reviewer, QA) to operate within a flat mesh network. By leveraging AutoGen's flexible group chat mechanics, agents communicate peer-to-peer, dynamically self-select the next speaker based on the context of the codebase, and must collaboratively reach consensus without a centralized adjudicator. This structure tests the system's susceptibility to "polite spiraling" and its capacity for emergent problem-solving.

By utilizing a shared codebase for both the hub-and-spoke and the mesh network, any variance in token consumption, task completion rate, or code correctness can be definitively attributed to the architectural shift rather than difference in agent capability or tool access.

3.2 Implementation

3.2.1 Agents

The implemented system is a multi-agent software engineering framework composed of four active roles: Project Manager, Engineer, Code Reviewer, and QA. These roles are instantiated as task-oriented agents with distinct responsibilities, tool access, and execution constraints. The two implemented variants share the same execution baseline and role definitions at a high level, while differing mainly in how responsibilities are routed between roles. Since architectural differences are discussed separately, this section focuses only on the implemented role behaviors and supporting runtime mechanisms.

3.2.1.1 Project Manager

The Project Manager is implemented as the coordination and task-state management layer of the system. It is not responsible for modifying source code directly; instead, it translates the incoming user requirement or SWE-bench issue into structured work

items and coordinates the remaining agents through explicit handoff tools. In the implemented workflow, the Project Manager receives the initial task, creates one or more tickets on the project board, delegates work to the Engineer, Code Reviewer, and QA, and synthesizes the final completion summary once the task has been validated.

A central implementation feature of this role is the project-board mechanism. For each request, the Project Manager is instructed to create separate ticket files under data/project_board/tickets/ and maintain a lightweight index at data/project_board/index.md. Each ticket records fields such as ID, title, status, owner, description, acceptance criteria, tasks, links, and update history. Ticket status follows a simple Kanban-style workflow: TO DO, IN PROGRESS, IN REVIEW, and DONE. This gives the system a persistent coordination state outside the conversation history, allowing downstream agents to understand scope, acceptance criteria, and prior progress without relying only on transient chat context.

The Project Manager also acts as a control point for workflow routing. In the centralized architecture, all specialist agents return to the Project Manager after completing their stage, making it the hub through which implementation, review, and validation are sequenced. In the decentralized architecture, the same role still performs initial decomposition and final completion, but some specialist-to-specialist routing is allowed after the task has entered the implementation pipeline. This distinction makes the Project Manager especially important in the centralized system, where it enforces ordering and prevents agents from advancing to later stages without explicit coordination.

For SWE-bench execution, the Project Manager is extended with benchmark-specific rules. It is instructed to treat each task as a minimal bug-fix workflow rather than a greenfield software project, avoid creating test-update tickets by default, and assume that the benchmark harness has already applied the gold test patch. The Project Manager must also inspect structured validation evidence before declaring

completion. In particular, it must confirm that the Engineer produced a non-empty patch, that QA emitted a QA_VERDICT with verified: true, and that no Code Reviewer verdict reports blocking findings.

This implementation makes the Project Manager both a planner and a completion gate. Its role is not merely to distribute work, but to preserve traceability between the issue statement, the ticketed acceptance criteria, the implementation patch, and the final verification result. By separating coordination from code modification, the system reduces the chance that planning decisions and implementation edits become conflated within a single agent.

3.2.1.2 Engineer

The Engineer is the most technically specialized component of the system. It is not implemented as a plain conversational agent alone, but as a wrapper around an inner mini-swe-agent [17] execution loop. As illustrated in the detailed architecture flowchart in Figure 3.1, each time the Engineer role is invoked, the system launches a fresh inner execution session, supplies the relevant task context, allows the model to act through tool-mediated commands, and then returns a structured result summary to the outer multi-agent runtime.

Several implementation choices distinguish this role from a generic LLM agent. First, the default prompt behavior of the inner agent is modified so that execution must occur through explicit shell-tool calls rather than informal markdown command formatting. Second, the Engineer supports controlled access to external tools through an MCP bridge, enabling interaction with the project board, documentation store, workspace, and repository utilities. Third, the implementation supports both local and containerized execution environments, allowing the same engineer logic to operate either on a host workspace or inside an isolated repository environment.

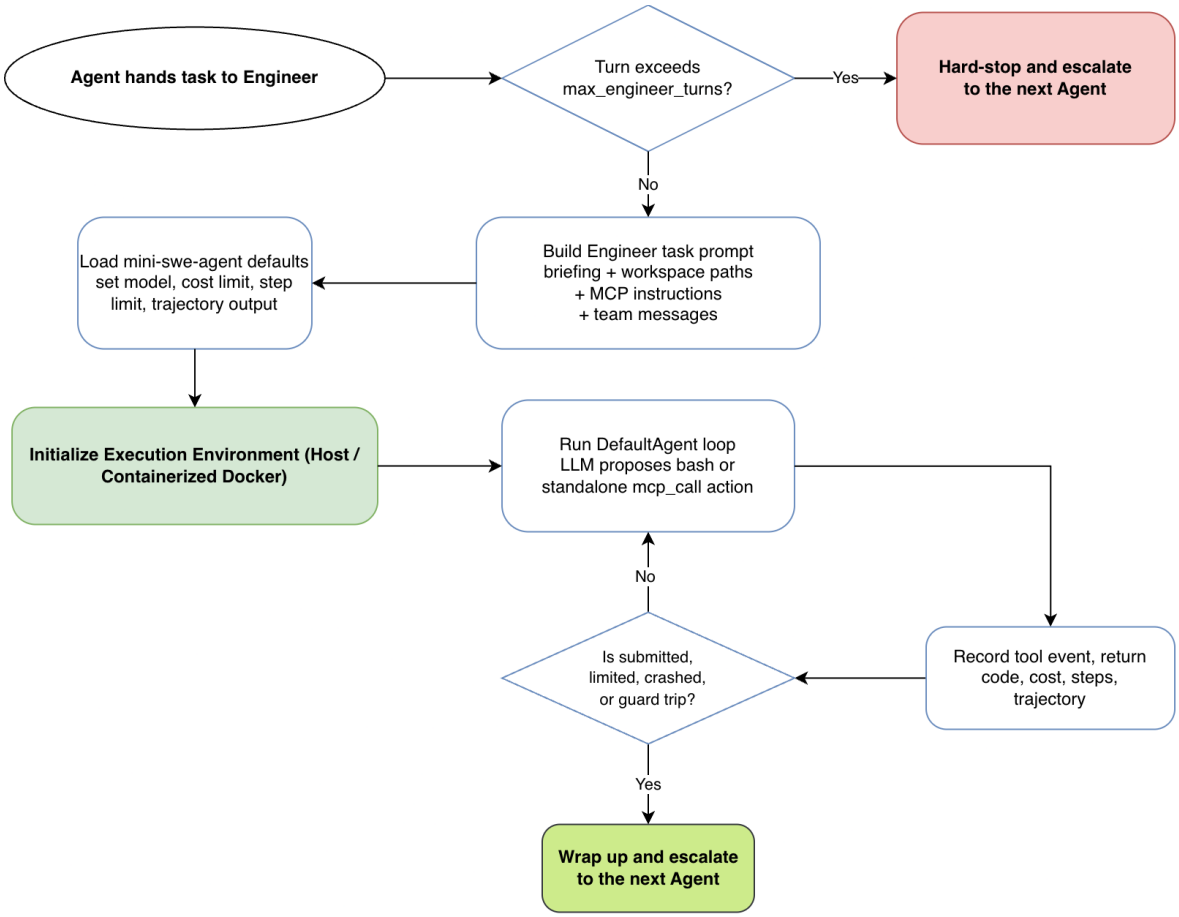


Figure 3.1. Flowchart illustrating the detailed implementation of the Engineer Agent, including both execution approaches.

The Engineer implementation also contains multiple layers of defensive mechanisms intended to improve reliability across three distinct scopes:

- Runtime and Swarm Limits: To prevent infinite billing or conversational loops, a strict counter tracks allocations before each run, applying absolute cost and step limits. If the agent's total turn count exceeds max_engineer_turns, or if a swarm-level loop guard detects a single agent monopolizing the conversation, the system executes a hard-stop and escalates the failure back to the ProjectManager.

- Environment and Tool Boundaries: Security and predictability are enforced by constraining the workspace scope. In local mode, paths and environment variables (PATH) are rigidly scoped, while in containerized mode, the execution runs entirely inside an official SWE-bench /testbed Docker image with the primary run directory

mounted as read-only. For tool interactions, the Model Context Protocol (MCP) bridge enforces a strict allowed-server list, mandates that mcp_call must encapsulate the entire action, and rejects execution if the accompanying JSON arguments fail to parse into a valid object.

- Execution Loop Guardrails: Inside the inner command execution loop, a specialized LoopGuard actively monitors commands to halt destructive or stalled behaviors. The guard automatically trips and terminates execution if it catches the identical command being repeated 4 times, or if it encounters 12 consecutive non-zero exit codes.

Finally, the Engineer generates a patch artifact and logs full execution trajectories, which can be reviewed post-process to analyze its behavior and outputs.

3.2.1.3 Code Reviewer

The Code Reviewer is implemented as an independent inspection agent positioned between implementation and final validation. Its purpose is to evaluate whether the Engineer's patch is correct, minimal, maintainable, and aligned with the issue requirements. Unlike the Engineer, the Code Reviewer is not allowed to modify implementation files. Its contribution is instead expressed through diagnostic feedback, ticket updates, routing decisions, and a structured review verdict.

Several implementation choices distinguish this role from a generic review prompt. First, the reviewer is given scoped read access to the project board and documentation so that it can compare the patch against the intended task rather than inspecting the diff in isolation. Second, in local execution mode, it can inspect implementation files and git state directly through code-reading and git tools. Third, in Docker-based SWE-bench mode, where the modified repository exists inside the benchmark container rather than the host workspace, the reviewer relies on patch-oriented tools such as read_patch_diff. This ensures that the review is performed against the actual patch generated by the Engineer rather than an empty or stale host-side workspace.

The Code Reviewer is also constrained by SWE-bench-specific review rules. It must inspect the unified diff before approving or rejecting a change, focus on correctness, regression risk, and unnecessary scope expansion, and avoid requiring test-file edits unless explicitly required by the task. This is important because the SWE-bench harness applies the gold test patch automatically; therefore, edits to tests are usually treated as an incorrect repair strategy. The reviewer is instructed to approve only when the patch directly addresses the stated bug with minimal scope and to reject patches that are incomplete, overly broad, unrelated, or likely to introduce behavioral side effects.

The role produces a machine-readable REVIEW_VERDICT block at the end of inspection. This block records whether blocking findings are present and includes a short summary of the review. The runtime parses this verdict and uses it as a control signal: if blocking_findings is true, the workflow is not considered valid for completion. In the centralized topology, the reviewer returns control to the Project Manager after inspection. In the decentralized topology, it may route directly to QA when the implementation is acceptable, or back to the Engineer when blocking issues remain.

Thus, the Code Reviewer functions as a quality gate between generated code and behavioral testing. Its implementation reflects the assumption that passing responsibility from generation to validation should not happen automatically; the patch must first be checked for scope, relevance, and maintainability. This role is especially important for detecting cases where a patch appears locally plausible but changes more of the codebase behavior than the task requires.

3.2.1.4 Quality Assurance

The QA agent is implemented as the system's behavioral validation component. While the Code Reviewer evaluates the structure and appropriateness of the patch, QA determines whether the implementation actually satisfies the observable task requirements. It is responsible for inspecting the ticket context, running targeted

verification commands, documenting test outcomes, and reporting whether the task can be considered resolved.

The QA implementation is built around tool-mediated validation. It has access to the project board and shell-based command execution tools, allowing it to read ticket requirements, run relevant tests, and record results back into the workflow state. In the general workflow, QA is instructed to design tests that cover happy paths, edge cases, and regressions. In the SWE-bench workflow, this behavior is narrowed into a benchmark-specific protocol focused on the provided fail-to-pass and pass-to-pass tests.

For SWE-bench tasks, QA must first perform test discovery before running targeted tests. It is instructed to use pytest collection to confirm that the listed fail-to-pass test IDs exist, then run those fail-to-pass tests directly, and only broaden validation after the targeted tests are collected and exercised. This prevents the system from incorrectly treating an unrelated full-suite result as evidence that the benchmark issue has been resolved. It also prevents the Engineer from being asked to add missing tests, because the benchmark harness is expected to apply the gold test patch before validation begins.

The QA role produces a structured QA_VERDICT block as its final validation output. This block contains a verified field, a list of failing or missing test IDs, and a short summary of the validation result. The runtime parses this block and requires verified: true before the run can be treated as complete. If the verdict is missing, if verified is false, or if targeted fail-to-pass tests are absent or failing, the workflow is marked as incomplete or failed verification.

This implementation makes QA the strongest outcome-based control point in the system. A patch cannot be accepted solely because it was produced by the Engineer or approved by the Code Reviewer; it must also be supported by explicit test evidence. As a result, QA connects the internal multi-agent workflow to the

external benchmark objective, ensuring that completion depends on observed behavioral correctness rather than conversational agreement among agents.

3.2.1.5 Shared Runtime Support

All active roles operate on top of a shared runtime layer that provides controlled access to files, repository state, execution context, inter-agent communication, and internal bookkeeping. Agent-to-agent communication is handled by AutoGen's in-process asynchronous group-chat runtime. Therefore, the agents do not communicate with one another through HTTP, gRPC, or JSON-RPC as a network protocol. Instead, each role is registered as an AutoGen participant, and the runtime passes structured Python message objects between agents, such as normal text messages and explicit handoff messages.

The standard communication protocol between agents is based on AutoGen handoff control. When an agent completes its current responsibility, it does not directly invoke another agent through a remote API call. Instead, it emits a structured HandoffMessage through a permitted handoff tool, such as transfer_to_Engineer, transfer_to_CodeReviewer, transfer_to_QA, or transfer_to_ProjectManager. AutoGen then updates the active speaker and transfers the conversation state to the target agent. This makes role transitions explicit and traceable while keeping the communication inside the same Python runtime.

A separate protocol is used for tool access. The agents' filesystem, documentation, workspace, and git operations are exposed through Model Context Protocol servers. In this implementation, MCP manages agent-to-tool communication through JSON-RPC over standard input and output. The runtime launches MCP servers as subprocesses, such as filesystem servers for scoped directory access and a git server for repository inspection. The implementation does not use HTTP or gRPC for internal agent communication. HTTP-based communication may occur only indirectly when the underlying model client contacts the external LLM provider, but that is separate from the multi-agent coordination protocol itself.

As illustrated in the directory layout of Figure 3.2, the system root (data/ or data_/) separates the asset domains into three distinct subdirectories, each exposed to the multi-agent system via a dedicated Model Context Protocol (MCP) scoped root:

- fs_board: Provides a scoped interface to the project_board/ directory, exposing tracking tickets, system status, and the central task index.
- fs_docs: Provides a scoped interface to the knowledge_base/ directory, containing design notes, system requirements, and architecture documentation.
- fs_code + git: Provides a scoped interface to the active development workspace/, granting direct access to the implementation repository, test suites, and repository inspection tools like git diffs and status information.

Thus, the system uses a layered communication model. AutoGen manages agent-to-agent coordination through in-process message passing and handoff events, while MCP manages agent-to-tool communication through JSON-RPC over standard input and output. The implementation does not use HTTP or gRPC for internal agent communication. HTTP-based communication may occur only indirectly when the underlying model client contacts the external LLM provider, but that is separate from the multi-agent coordination protocol itself.

3.2.2 Multi-Agent Coordination Architectures

The proposed system is implemented in two coordination variants that share the same active agent set as mentioned above. The distinction between the two variants does not lie in the existence of different agent roles, but in how control over task progression is distributed among them. In both cases, the Project Manager is responsible for high-level task framing, the Engineer performs implementation, the Code Reviewer assesses the produced changes, and QA validates the resulting behavior. What changes between the two variants is the degree to which specialist agents are allowed to influence the next stage of the workflow.

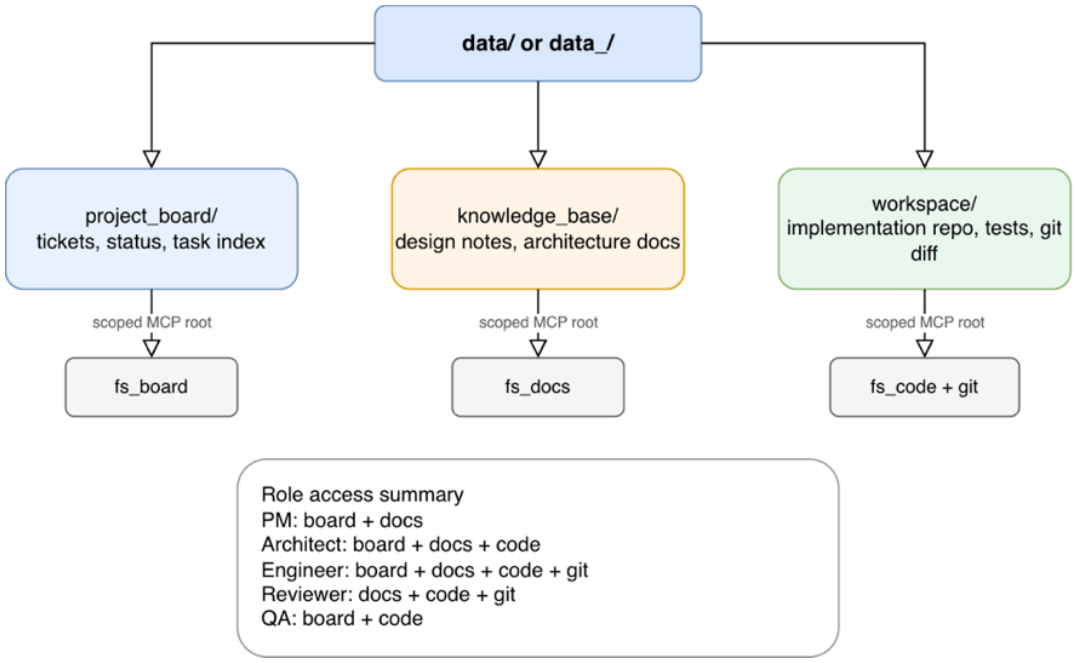


Figure 3.2. Diagrams represents the working data directory structure of the agent workspace, showing scoped storage areas and role-specific access paths.

From this perspective, the centralized and decentralized variants can be understood as two alternative coordination architectures built on top of a shared multi-agent foundation. The centralized variant emphasizes hierarchical control through a single managerial hub, whereas the decentralized variant introduces limited autonomy into the technical workflow by allowing certain specialist agents to participate in local routing decisions. These two forms provide a useful basis for examining how different distributions of control shape multi-agent software engineering behavior.

3.2.2.1 Centralized Coordination

In the centralized coordination architecture as shown in Figure 3.3, the Project Manager acts as the single control center of the workflow. All major task transitions are mediated through this role. After the Project Manager interprets the incoming requirement and decomposes it into actionable work, implementation is assigned to

the Engineer. Once the Engineer completes its stage, the output is returned to the Project Manager, which then determines whether the next step should be code review, validation, revision, or final completion. The same pattern applies to the outputs of the Code Reviewer and QA, both of which report back to the Project Manager rather than advancing the workflow independently.

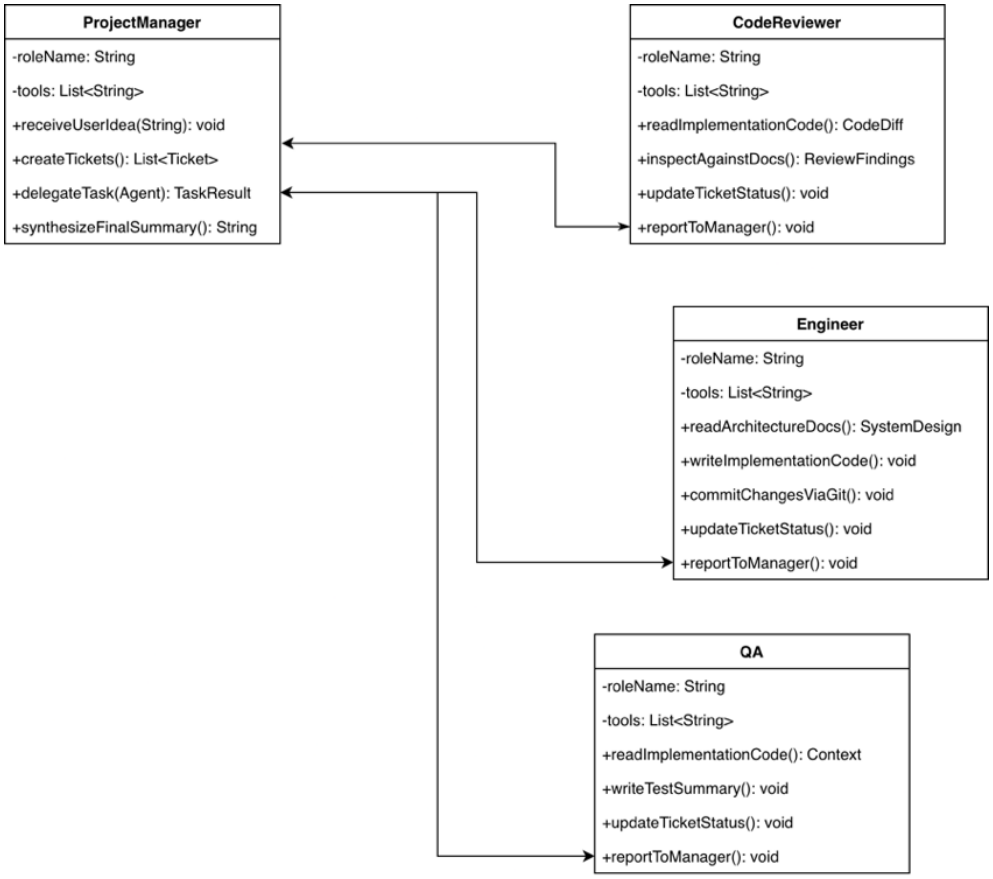


Figure 3.3. Class diagram illustrating the centralized Multi-Agent System, highlighting the strict hub-and-spoke communication pathways routed through the Project Manager node.

This structure creates a hub-and-spoke coordination model. Specialist agents operate as bounded functional units that perform implementation, inspection, or validation, but they do not control the global direction of the task. Workflow

ownership therefore remains concentrated in the managerial role. In term of architecture, this design leans over explicit oversight, clear accountability, and unified interpretation of current project state. Since all intermediate results are funneled through one central role, the system maintains a single authoritative perspective on task status and next-step decisions.

The main advantage of centralized coordination is its procedural clarity, thanks to the Project Manager's responsibility for all major transitions, which also means the workflow is easier to track, interpret, and measure. This can be beneficial in complex software engineering tasks where maintaining consistency across multiple stages is important. At the same time, the architecture places strong dependence on the central role, since the quality of task progression is closely tied to the Project Manager's ability to interpret specialist outputs and make effective routing decisions.

3.2.2.2 Decentralized Coordination

In the decentralized coordination architecture, as illustrated in Figure 3.4, the same four active roles are preserved, but selected elements of workflow control are delegated to specialist agents. The Project Manager still provides task framing, progress supervision, and final synthesis, but it no longer has to mediate every technical transition. Instead, some decision authority is distributed into the review and validation stages.

Under this arrangement, the Code Reviewer is not limited to reporting findings upward. It can determine whether the implementation should proceed toward validation or return for revision. Similarly, QA can direct the workflow toward either completion or further correction based on the observed validation outcome. As a result, the architecture supports local technical loops that can operate with reduced managerial intervention. This allows technical agents to respond more directly to the evidence generated within their own stage of work.

The decentralized architecture is therefore only partially distributed rather than fully peer-to-peer. The Project Manager remains the overall supervisory role and

continues to maintain responsibility for high-level state and final completion. However, by allowing specialist agents to resolve certain intermediate transitions themselves, the architecture reduces the number of occasions on which all workflow decisions must be re-centralized. Compared with the centralized variant, this produces a more flexible and locally adaptive coordination structure.

The main advantage of this approach is that technical correction cycles can proceed more directly. Review findings and validation failures do not always need to be routed back through a central coordinator before the next appropriate action is taken. This can reduce coordination overhead and improve responsiveness inside iterative development loops. However, the architecture also depends more heavily on specialist judgment, because local routing decisions are no longer made exclusively by the managerial role.

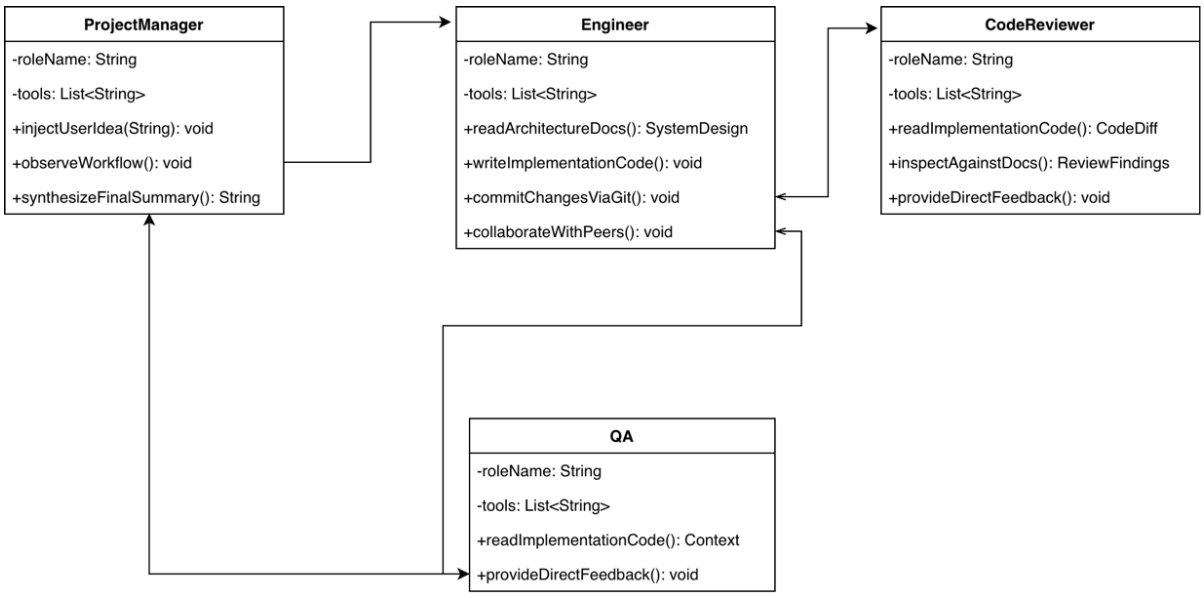


Figure 3.4. Class diagram illustrating the decentralized Multi-Agent System, demonstrating the removal of Manager-dependent delegation in favor of direct feedback loops and inter-agent debate.

3.3 Evaluation Pipeline

The evaluation pipeline provides the experimental framework used to benchmark the proposed agent systems in a consistent and reproducible manner. Its purpose is to ensure that all evaluated systems operate under the same task definitions, repository states, verification conditions, and artifact-collection procedures. Rather than being tied to one specific agent architecture, the pipeline is designed as a reusable benchmarking layer that can execute multiple systems through a common process.

In this study, the pipeline is used to evaluate the centralized and decentralized multi-agent systems, together with a single-agent baseline, under the same benchmark protocol. It covers the full experimental lifecycle, including benchmark instance loading, task selection, workspace provisioning, execution, and output recording. This makes it the main infrastructure that connects the conceptual agent designs to their empirical assessment.

3.3.1 Manifest Construction

To manage benchmark execution systematically, the pipeline organizes tasks into manifests as illustrated in Figure 3.5. A manifest is a structured execution plan that records which benchmark instances will be run, from which dataset and split they were drawn, and what selection strategy was used. This intermediate representation separates task selection from task execution, improving both clarity and repeatability.

The pipeline supports two selection modes. The first uses an explicit list of benchmark instance identifiers, allowing the experimenter to define an exact evaluation subset. The second uses an automatic selection strategy that distributes tasks across repositories in a balanced round-robin manner. This approach is useful when constructing a compact benchmark set while still preserving diversity across repository families.

Once built, a manifest can be saved and reused across repeated trials or across multiple systems. This ensures that the same benchmark population is used in the comparative experiments, which is necessary for a fair comparison. In this sense, the

manifest acts as a reproducible contract between benchmark preparation and system execution.

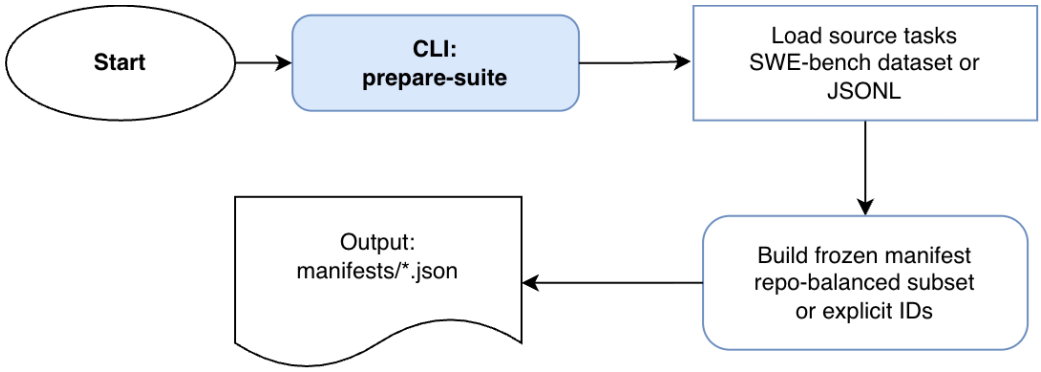


Figure 3.5. Flowchart of the benchmark suite preparation pipeline, illustrating the workflow from loading source tasks to generating a frozen manifest artifact via explicit ID selection or repository-balanced subsetting.

3.3.2 Workspace Preparation

Before any system can be executed on a benchmark instance, the pipeline prepares a task-specific workspace as shown in Figure 3.6. This stage ensures that the correct repository state and execution environment are available for the run.

The pipeline supports two workspace-preparation strategies. In host mode, it creates a local working copy of the target repository, checks out the specified base commit, and optionally, provides a compatible runtime environment. In Docker mode, it instead uses the official SWE-bench per-instance Docker image, which already contains the repository and environment in a form aligned with the benchmark expectations.

When host-mode preparation is used, the pipeline can automatically create a virtual environment and install a set of common build dependencies before performing editable installation of the target repository. This is intended to improve compatibility with repositories that depend on older packaging conventions, historical dependency sets, or compiled extensions. In contrast, Docker-mode

preparation avoids local environment drift by using the benchmark container as the authoritative execution environment.

The workspace-preparation stage also records task metadata so that the execution phase can access a stable representation of the benchmark instance. As a result, the system being evaluated does not need to query the original benchmark source directly during runtime; instead, it receives a prepared and localized task context.

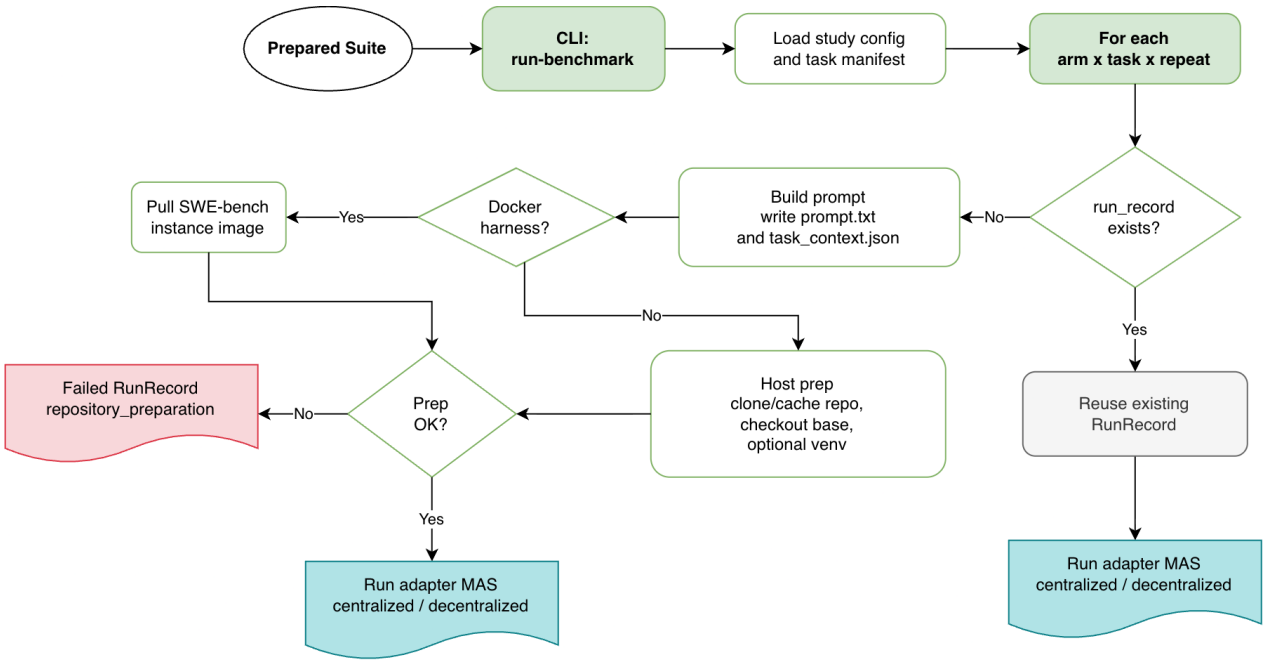


Figure 3.6. Flowchart illustrating the execution pipeline initiated via run-benchmark, highlighting the conditional paths for Docker and host-mode workspace preparation strategies

3.3.3 Execution Adapters

Figure 3.7 demonstrating the pipeline that executes benchmarked systems through an adapter abstraction. This design allows different systems to be evaluated under a uniform framework while preserving their individual invocation requirements. In the current setup, the adapter layer supports three execution arms: the centralized multi-agent system, the decentralized multi-agent system, and a standalone mini-swe-agent baseline.

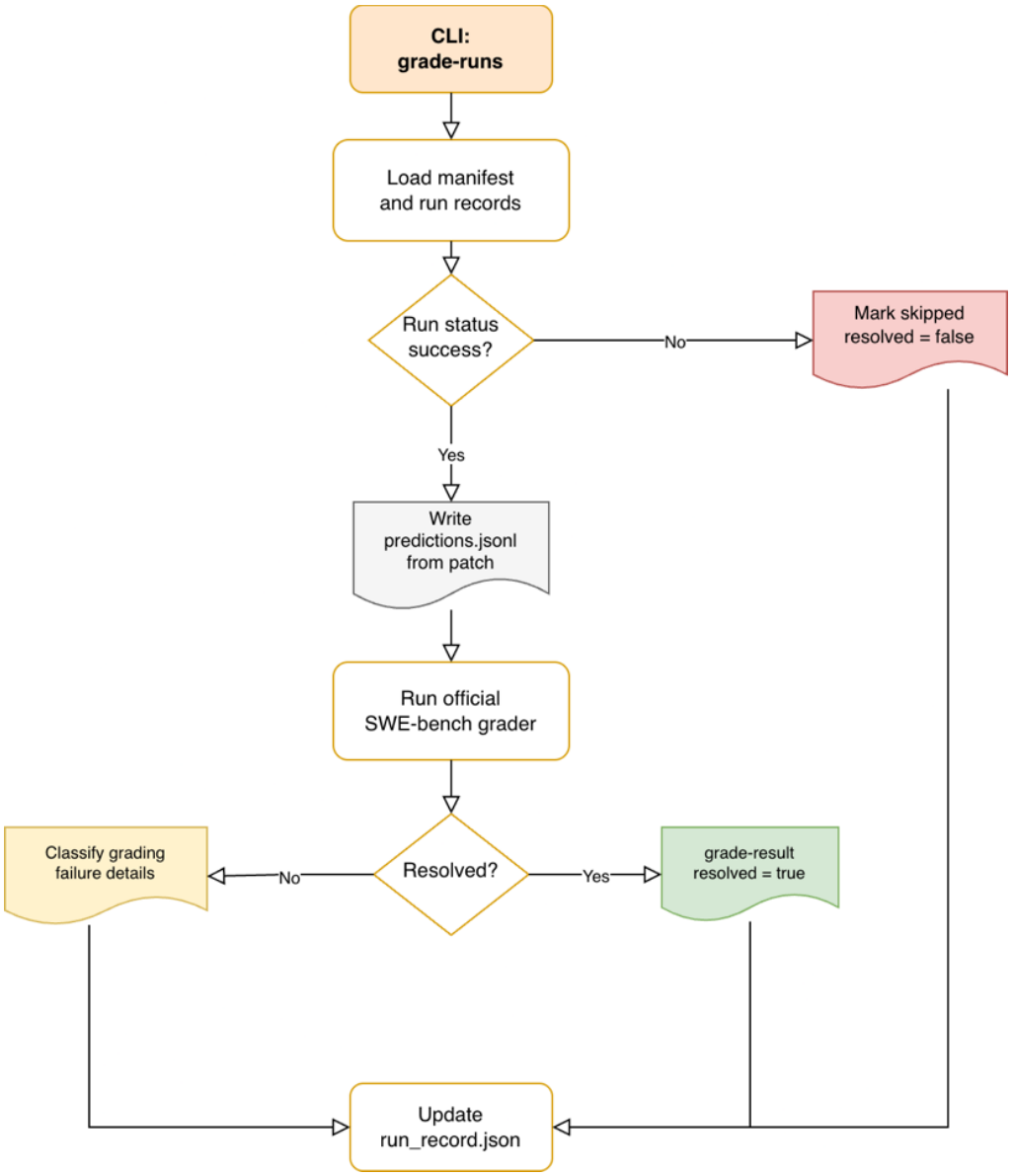


Figure 3.7. Flowchart of the evaluation grading pipeline initiated by grade-runs, detailing the logic from loading run records and writing predictions to executing the official SWE-bench grader and updating run_record.json based on resolution status

A common command-backed adapter is used to launch each system. For every run, the adapter creates a dedicated run directory, prepares the necessary paths, and injects standardized environment variables that expose the workspace location, task context, telemetry target, result file path, and patch output location. This guarantees that all systems receive benchmark information through the same operational interface.

The adapter layer also manages the distinction between full-container execution and hybrid execution. In the case of the single-agent baseline, the whole process may run inside the SWE-bench Docker environment. In the case of the multi-agent systems, the outer coordination process remains on the host, while implementation-level commands generated by the Engineer may be redirected into the Docker-backed benchmark workspace. This hybrid design preserves the agent orchestration logic while still allowing repository interaction to occur in the benchmark environment expected by SWE-bench Lite.

An additional responsibility of the adapter layer is to provide the benchmark-supplied test patch when it exists. The test patch is written into the run directory and made available during execution so that verification can proceed with the intended fail-to-pass and pass-to-pass tests already in place. This ensures that the evaluated systems are assessed on implementation repair rather than on reconstructing benchmark tests.

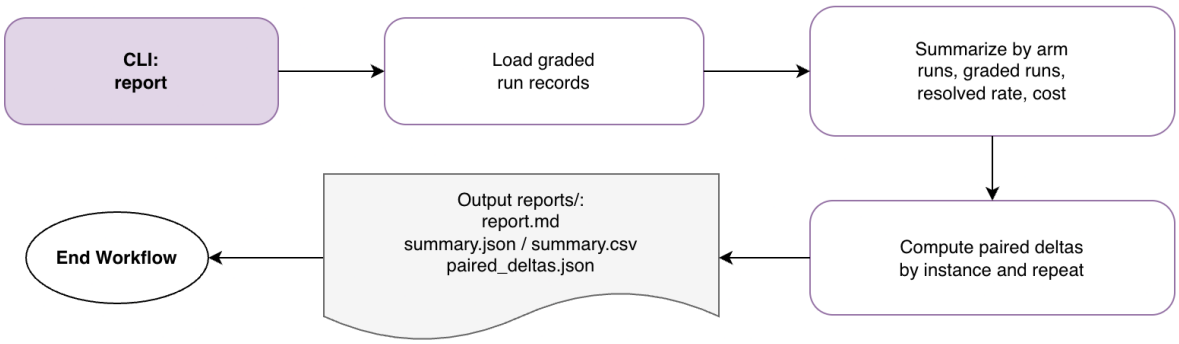


Figure 3.8. Flowchart of the reporting pipeline initiated via CLI: report, illustrating the sequential process of loading graded run records, summarizing operational metrics by arm, computing paired deltas, and generating the final markdown and structured data outputs

3.3.4 Artifact Collection and Reproducibility

For every benchmark run, the pipeline records a structured set of artifacts that document both the execution process and its outcome. These artifacts typically include the generated prompt, task context, standard output, standard error, the produced patch file, telemetry data, and a structured result summary. Together, they make each run inspectable after completion.

The pipeline in Figure 3.8 also records a formal run record containing execution status, timing information, failure categories, and links to the generated artifacts. This allows each run to be treated as a traceable experimental unit rather than as a transient command invocation. In addition, if a run record already exists, the pipeline can skip re-execution, which avoids unnecessary duplication and preserves prior results.

Telemetry plays an important role in this design. Beyond recording whether a benchmark instance was solved, the pipeline also captures process-level indicators such as token usage, tool-call counts, tool failures, retries, handoffs, and message counts. This supports richer analysis of system behavior and allows the evaluation to consider not only final correctness but also operational efficiency and execution characteristics.

The evaluation pipeline is designed for maximum overall reproducibility. With explicit manifests, structured workspaces, standardized adapters and persistent artifact directories, we ensure that benchmark runs can be repeated, audited and compared across systems under stable conditions, For a study based on SWE-bench Lite, this reproducibility is essential because it ensures that observed differences between systems can be attributed to the systems themselves rather than to changes in task selection or execution setup.

CHAPTER 4. EXPERIMENT

This section describes the experimental procedure used to evaluate the two proposed multi-agent software engineering systems. The experiment uses a fixed benchmark dataset, a shared execution pipeline, and the same grading procedure for both systems. The results reported in this section provide the empirical basis for the discussion in the following chapter.

4.1. Benchmark Dataset

Within the evaluation pipeline, each SWE-bench Lite instance is processed as a structured task record. This record preserves the original engineering context by storing the following components:

- Repository Identity and Base Commit: Establishes the exact baseline and version control state of the codebase.
- Problem Statement: A natural-language description extracted directly from a real user issue report.
- Testing Criteria: Evaluation relies on two distinct groups of test identifiers to verify the repair outcome:
- Fail-to-pass tests: Tests that are failing on the base commit but are expected to pass after a correct fix is applied.
- Pass-to-pass tests: Regression tests that must be continue to pass after changes are been made, to ensure existing functionality is not broken.
- Supplementary Metadata: Optional historical issue hints, environment setup configurations, and the benchmark-supplied test patch.

A task instance is considered successfully resolved only when a generated patch satisfies these strict grading criteria.

For this study, 50 instances were selected from the SWE-bench Lite test split. To prevent bias and avoid overrepresenting any single project or software domain, the evaluation subset was constructed using a repository-balanced round-robin

strategy. The resulting selection contains a diverse range of foundational open-source codebases, which includes Django, pytest, matplotlib, scikit-learn, Sphinx, SymPy, xarray, seaborn, Flask, Requests, Pylint, and Astropy.

Using this specific benchmark configuration offers two distinct advantages for the experimental design:

- Standardized Benchmarking, where this dataset already provides an externally recognized, reproducible, and rigorous framework for evaluating repository-level code repair systems.
- Realistic Engineering Constraints, showing by the architectural variety across the selected repositories. This would allow the proposed systems to work with various codebase structures, testing conventions, and environment dependencies. Success reflects a comprehensive capacity for repository understanding, environment handling, and targeted test validation rather than mere code generation.

4.2. Experimental Setup

The experiment compares two multi-agent system variants: the centralized MAS and the decentralized MAS. Both systems were evaluated using the same benchmark instances, model configuration, timeout limit, execution environment, and grading process. This setup ensures that the primary experimental difference is the communication structure between agents rather than external factors, such as dataset selection or evaluation procedure.

The centralized system follows a hub-and-spoke coordination pattern, where the Project Manager acts as the main coordinator between specialist agents. The decentralized system follows a swarm-based coordination pattern, where agents can hand off work directly to one another according to the current task state. The implementation details of these systems were presented in the previous section, so this experiment treats them as two evaluated arms of the same benchmark pipeline.

For each benchmark instance, the evaluation pipeline prepared the task context, executed the corresponding MAS implementation, collected the generated patch, and submitted the patch to the SWE-bench grading harness. The systems were run under the same timeout limit of 1,800 seconds per task. Each task was executed once for each system. The generated artifacts, including prompts, logs, patches, telemetry, and grading results, were stored for later inspection.

A single-agent baseline using gpt-5-mini was also included where baseline grading information was available. However, because the baseline results were incomplete for many of the selected instances, they are reported separately and used only as supporting context rather than as the primary comparison.

4.3. Evaluation Metrics

The main evaluation metric is resolved rate, defined as the percentage of graded benchmark instances successfully resolved by a system. A resolved task is one where the generated patch passes the SWE-bench grading process. This metric directly measures functional correctness with respect to the benchmark tests.

More on resolved rate, the experiment also reports the number of resolved tasks, average execution duration, and average token usage. Execution duration measures the wall-clock time required for a system to complete a task. Token usage measures the total number of model tokens consumed during the run and provides an estimate of computational cost. These efficiency metrics are important because multi-agent systems may improve task-solving performance at the cost of additional coordination overhead.

The experiment also uses paired task-level comparison between the centralized and decentralized systems. Since both systems are evaluated on the same 50 benchmark instances, each task can be compared directly. This makes it possible to identify cases where both systems succeed, both fail, or only one system resolves the task.

4.4. Results

The centralized MAS resolved 35 out of 50 benchmark instances, corresponding to a resolved rate of 70.00%. The decentralized MAS resolved 33 out of 50 instances, corresponding to a resolved rate of 66.00%. Both systems were successfully graded on all 50 tasks. Details are shown in Table 4.1 and Table 4.2, with comparison in Figure 4.1.

In terms of efficiency, the centralized MAS required an average of 584.36 seconds per task and consumed an average of 433,991 tokens. The decentralized MAS required an average of 443.54 seconds per task and consumed an average of 551,079 tokens. Thus, the decentralized system completed tasks slightly faster on average, while the centralized system used fewer tokens on average and achieved a higher resolved rate.

The paired comparison shows that the two systems produced the same resolved or unresolved outcome on 44 of the 50 tasks. The decentralized MAS resolved 2 tasks that the centralized MAS did not resolve, while the centralized MAS resolved 4 tasks that the decentralized MAS did not resolve. This indicates that the two architectures perform similarly on most benchmark instances, but differ on a small number of task-specific cases.

The gpt-5-mini baseline had available grading results for 11 of the 50 benchmark instances. Among those graded cases, it resolved 7 tasks, resulting in a resolved rate of 63.64% on the available subset. Because 39 baseline cases were missing or unknown, this result should not be interpreted as a complete comparison against the MAS variants.

Overall, the experimental results show that both multi-agent systems are capable of resolving a substantial portion of the selected SWE-bench Lite tasks. The centralized MAS achieved the highest resolved rate in this experiment, while the decentralized MAS showed comparable performance with a slightly lower average execution time. The implications of these results are discussed in the next section.

Table 4.1: Comparative evaluation of Centralized MAS, Decentralized MAS, and baseline configurations across execution duration, token consumption, and resolution rates.

Architecture / Baseline	**Resolved Tasks**	**Resolved Rate**	**Avg. Execution Time**	**Avg. Token Consumption**
Centralized MAS	35 / 50	70.00%	584.36 seconds	433,991 tokens
Decentralized MAS	33 / 50	66.00%	443.54 seconds	551,079 tokens

Table 4.2: Paired comparison and topological convergence dynamics between Centralized MAS and Decentralized MAS configurations.

Comparison Metric	**Task Count (out of 50)**	**Percentage**
Identical Outcome (Both Resolved / Both Unresolved)	44	88.00%
Uniquely Resolved by Centralized MAS	4	8.00%
Uniquely Resolved by Decentralized MAS	2	4.00%

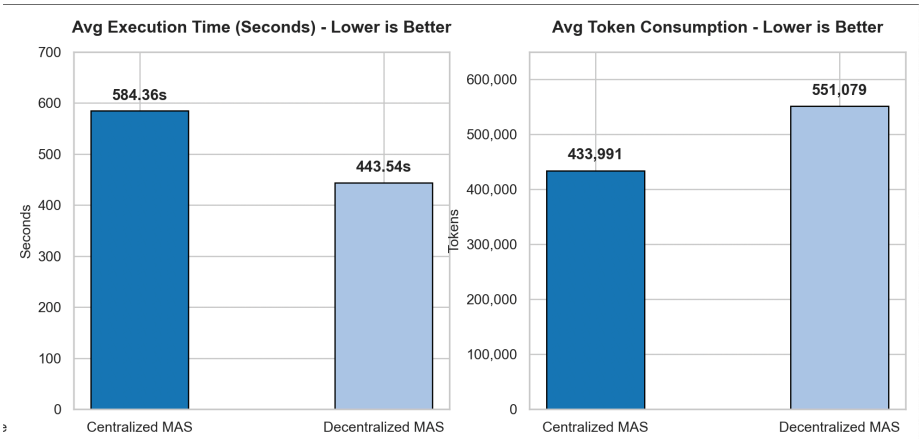


Figure 4.1. Comparative evaluation of Centralized MAS, Decentralized MAS across execution duration, and token consumption.

CHAPTER 5. DISCUSSION

This section discusses the experimental results presented in the previous section. The purpose of this discussion is to interpret the observed performance differences between the centralized and decentralized multi-agent systems, rather than simply restating the numerical results. The discussion focuses on how agent communication structure may have influenced correctness, efficiency, and practical usability in automated software engineering tasks.

5.1 Experimental Results

The narrow performance variance in final task resolution rates underscores how profoundly agent communication topology governs system reliability and resource allocation, rather than absolute capability. Instead of one architecture proving universally superior, the empirical data exposes a clear, fundamental trade-off between centralized control and decentralized flexibility.

The centralized MAS leverages its hub-and-spoke topology to enforce a rigid, highly predictable control flow. By positioning the Project Manager as the sole intermediary for all information exchanges, the system minimizes role ambiguity and prevents tasks from progressing prematurely. For long-horizon software engineering workflows—which demand a systematic sequence of issue interpretation, patch generation, and validation—this structural gatekeeping preserves a cohesive global state. Sub-agents are blocked from operating under conflicting assumptions, resulting in narrower patch precision, strict requirement coverage, and superior regression containment.

In contrast, the decentralized MAS optimizes for operational fluidity by allowing specialized agents to engage in direct, peer-to-peer handoffs. Stripping away the processing and routing latencies of a central managerial hub allows the system to transition more naturally between design, implementation, and review cycles. This structural agility directly accounts for the decentralized network’s lower wall-clock

execution duration, positioning it as a compelling option for development environments where response time is the primary constraint.

However, this operational flexibility introduces severe systemic vulnerabilities. Without an authoritative coordinator to arbitrate state transitions, the decentralized workflow exhibits extreme sensitivity to early handoff quality. A single subpar output or faulty routing decision by a specialist agent easily propagates downstream, inducing unrecoverable cascade failures because subsequent nodes lack a centralized reference point for error recovery. Furthermore, the decentralized network's elevated token consumption reveals that its execution velocity comes at a steep computational premium. Lacking a centralized state cache, peer-to-peer agents are forced to frequently expend tokens duplicating context across handoffs, negotiating state changes, and resolving alignment issues during conversational loops.

5.2 Task-Level Performance Differences

The aggregate resolved rates provide an overall comparison of the centralized and decentralized architectures, but they do not explain why their outcomes differed on individual tasks. A task-level analysis was therefore conducted using the generated patches, internal QA reports, SWE-bench grading results, and reference patches included in the benchmark dataset.

The analysis distinguishes between tasks with valid SWE-bench results and tasks for which the external evaluation could not be completed. Tasks affected by evaluation-environment failures were excluded from the quantitative comparison. Their patches were examined separately through qualitative comparison with the corresponding reference patches.

5.2.1. Paired Outcome Comparison

Among the 50 task pairs, 44 received valid SWE-bench evaluations for both centralized and decentralized architecture. The remaining six pairs contained an

external evaluation failure for at least one system and were not used to compare resolved outcomes.

Of the 44 valid pairs, both systems resolved 31 tasks, while neither system resolved 9 tasks. The centralized MAS alone resolved 3 tasks, whereas the decentralized MAS alone resolved 1 task. Consequently, both architectures produced the same outcome in 40 of the 44 valid comparisons, corresponding to an agreement rate of approximately 90.9%.

This high agreement indicates that communication topology did not determine the final outcome for most tasks. In many cases, both architectures identified similar implementation areas and produced patches with equivalent benchmark results. The centralized architecture demonstrated an advantage in three valid cases, but this difference was concentrated in a small portion of the evaluated tasks.

The result also suggests that task characteristics and implementation quality may have a stronger effect than communication structure alone. For many relatively localized defects, both workflows were capable of directing the Engineer toward an appropriate part of the codebase. The effect of the architecture became more visible when tasks required coordination between multiple requirements, careful control of patch scope, or reasoning about interactions with existing behavior.

5.2.2 Architecture-specific Successes

Table 5.1 shows the differences between the successes of the 2 architectures. The first centralized-only success was matplotlib__matplotlib-23299, which concerned Matplotlib's rc_context behavior. The issue occurred because restoring the previous backend after leaving the context could trigger backend reinitialization and remove existing figures. The centralized MAS produced a narrowly scoped patch that preserved the backend selected within the context while restoring the remaining configuration parameters. The patch passed the target test without introducing tracked regressions.

Table 5.1: Comparative analysis of divergent benchmark outcomes across centralized and decentralized multi-agent architectures, detailing affected domains, resolution status, and primary architectural mechanisms.

Task Identifier	**Affected Component / Domain**	**Centralized Outcome**	**Decentralized Outcome**	**Primary Architectural Mechanism**
matplotlib__matplotlib-23299	Context State Management (rc_context)	**Resolved**	Unresolved (Regressions)	Centralized coordination enforced narrow patch scope; Decentralized suffered from local scope creep.
sympy__sympy-11400	Multi-Requirement Code Printing	**Resolved**	Unresolved (Missing Requirement)	Centralized workflow effectively mapped cross-component logical dependencies.
sympy__sympy-11897	Condition Isolation (LaTeX Output)	**Resolved**	Unresolved (Regressions)	Centralized maintained global state rules; Decentralized over-generalized the local fix.
sympy__sympy-11870	Specialized Mathematical Functions	Unresolved	**Resolved**	Decentralized architecture successfully routed the task directly to localized, domain-specific logic.

The decentralized MAS interpreted the requirement more broadly. Its patch attempted to preserve multiple configuration changes made within the context and also modified warning behavior during parameter validation. Although the reported symptom was corrected, the patch caused regressions in test_Bug_2543 and test_rcparams_reset_after_fail. The centralized patch was therefore more successful because it restricted the behavioral change to the backend, which was the specific source of the issue.

The second centralized-only success was sympy__sympy-11400. This task required support for both relational expressions and the sinc function in SymPy's C code printer. The centralized patch implemented both requirements. The relational implementation was also necessary for printing the conditional expression used to represent sinc. Both fail-to-pass tests consequently passed.

The decentralized patch implemented the sinc printer but omitted relational-expression support. As a result, test_ccode_Relational remained failing. The absence of correct relational printing also prevented the sinc expression from producing the expected C code, causing test_ccode_sinc to fail. This case suggests that the centralized workflow developed a more complete representation of the dependencies between the task requirements.

The third centralized-only success was sympy__sympy-11897, which concerned bracket placement around Piecewise expressions in LaTeX output. The centralized patch introduced a Piecewise-specific condition while preserving the existing rules for integrals, products, and sums. This focused modification passed the benchmark tests.

The decentralized patch removed an existing positional condition for all of these expression types. Although this corrected the target Piecewise output, it also added brackets to unrelated expressions and caused test_latex_basic to regress. The difference between the outcomes demonstrates that a broader change is not necessarily more complete. In this case, the successful solution required distinguishing Piecewise from other expression types.

The decentralized-only success was sympy__sympy-11870, which involved rewriting sinc expressions. The centralized patch modified generic rewriting behavior in SymPy's Basic class. However, this did not implement the required representation of sinc at zero, and the target test remained failing.

The decentralized patch modified the sinc-specific rewrite method and returned a piecewise expression equal to sin(x)/x for nonzero values and one at zero.

This directly addressed the tested requirement. Unlike the other disagreement cases, the decentralized architecture arrived at the more appropriate domain-specific implementation point.

Together, these cases show that architecture-specific success was closely related to patch precision and requirement coverage. The centralized MAS performed better where multiple related requirements had to be coordinated or where a broad modification could introduce regressions. The decentralized MAS succeeded where a direct change to a specialized implementation provided the required behavior.

5.2.3 Common Failure Patterns

The nine valid tasks, as classified in Table 5.2, that neither architecture resolved can be divided into two main failure patterns. The first pattern consisted of incomplete fixes that left one or more fail-to-pass tests failing. This indicates that the generated patches did not fully satisfy the target behavioral requirements of the benchmark instance. This pattern occurred in django__django-11019, pylint-dev__pylint-6506, pylint-dev__pylint-7080, and sphinx-doc__sphinx-7686.

For django__django-11019, both systems attempted to modify Django's media-merging behavior, but neither captured the full ordering and deduplication semantics required by the benchmark. The centralized patch demonstrated greater partial progress, leaving 9 target tests failing compared with 15 for the decentralized patch. However, the remaining failures show that the task required more than a local syntactic adjustment. The correct behavior depended on preserving media ordering across multiple merge scenarios while avoiding unnecessary conflict warnings. This made the task sensitive to edge cases that were not fully handled by either system.

For pylint-dev__pylint-6506 and pylint-dev__pylint-7080, both systems modified shared configuration or module-discovery behavior. These areas are difficult because they are used by many command-line paths and test cases. A patch that appears to fix the reported issue can easily affect unrelated configuration parsing, option handling, or recursive discovery behavior. In both tasks, the systems addressed

part of the reported problem but failed the target tests and introduced additional regressions. This suggests that the agents were able to locate the relevant subsystem, but did not fully model the broader contract of that subsystem.

A similar issue appeared in sphinx-doc__sphinx-7686. Both systems attempted to filter imported members during autosummary generation, which was the correct general area of the codebase. However, the target tests and several regression tests remained failing. The task required careful distinction between local members, imported members, skipped members, and template-generation behavior. The generated patches appear to have addressed the visible symptom without fully preserving the expected autosummary semantics across all tested cases.

The second failure pattern consisted of patches that passed their target fail-to-pass tests but introduced regressions elsewhere. This pattern is particularly important because it shows that local success on the reported issue did not necessarily imply benchmark correctness. In these cases, the systems repaired the immediate symptom but changed shared behavior in a way that broke existing pass-to-pass tests.

Both architectures corrected the reported three-dimensional visibility behavior in matplotlib__matplotlib-23314, and both produced the same implementation. However, the patch caused 13 pass-to-pass tests to fail. This indicates that the change affected general axes behavior beyond the specific three-dimensional visibility case. The patch was therefore locally plausible but too broad in its effect on Matplotlib's rendering behavior.

Regression-based failures also occurred in psf__requests-1963, pytest-dev__pytest-11148, and sympy__sympy-12171. In each case, the systems produced a patch that addressed the reported behavior, but the modification interfered with existing assumptions elsewhere in the test suite. For psf__requests-1963, both systems modified redirect handling, but the resulting behavior caused regressions in existing Requests tests. For pytest-dev__pytest-11148, both systems passed the target import-mode tests but caused a namespace-package regression. For sympy__sympy-

12171, both systems addressed the target derivative-printing behavior but caused test_Pow to fail, suggesting that the change affected broader Mathematica printer behavior.

The psf__requests-2148 task contained a mixture of the two patterns. The centralized MAS passed the target tests but introduced a regression in test_mixed_case_scheme_acceptable. This means the patch handled the reported socket-error behavior but changed another part of Requests' expected behavior. The decentralized MAS, by contrast, left one fail-to-pass test still failing, specifically test_unicode_multipart_post. Thus, the centralized result was a regression failure, while the decentralized result was an incomplete-fix failure.

These failures reveal an important limitation in the shared QA strategy used by both architectures. Internal QA frequently focused on executing the fail-to-pass tests associated directly with the reported issue. In several cases, the QA agent declared the patch successful because the immediate target tests passed. However, SWE-bench later rejected the patch because pass-to-pass tests failed. This means that the systems sometimes optimized for resolving the visible issue without sufficiently checking whether the fix preserved existing behavior.

This discrepancy affected both the centralized and decentralized architectures, so it should not be interpreted primarily as a consequence of communication topology. Instead, it reflects a common weakness in the validation process. The agents treated fail-to-pass success as strong evidence of completion, while the benchmark required both fail-to-pass success and pass-to-pass preservation. In software engineering terms, the systems were often effective at producing local repairs but less reliable at ensuring regression safety.

A stronger QA stage would therefore need to validate both dimensions of correctness. At minimum, the QA agent should execute all available fail-to-pass tests and the relevant pass-to-pass tests before declaring a task complete. When running the full pass-to-pass set is too expensive, the system could select regression tests

based on changed files, imported modules, and known dependency relationships. For example, patches to shared configuration, printing, routing, rendering, or parsing logic should trigger broader regression validation because these components are likely to affect many behaviors beyond the immediate issue.

Overall, the unresolved tasks show that the main challenge was not only finding the correct file or producing a plausible patch. The harder problem was preserving the full behavioral contract of the modified subsystem. This explains why both architectures could pass local QA yet still fail SWE-bench grading. The result supports the broader conclusion that future MAS-based software engineering systems should improve regression-aware validation, not only agent coordination.

Table 5.2: Case analysis and taxonomy of shared failure modes across centralized and decentralized multi-agent architectures, categorized by task identifier, test status, regression impact, and primary failure mechanism.

Failure Category	Task Identifier	Target Test Status	Regression Impact	Primary Failure Mechanism
Incomplete Execution	django__django-11019 pylint-dev__pylint-6506 pylint-dev__pylint-7080 sphinx-doc__sphinx-7686	Remaining Failures	Minimal	Patch addressed the initial error symptom but failed to resolve complex downstream edge cases or deduplication rules.
Regression Introduction	matplotlib__matplotlib-23314 psf__requests-1963 pytest-dev__pytest-11148 sympy__sympy-12171	Passed	Extensive (e.g., 13 failures in matplotlib)	Plausible local code corrections altered shared utility behaviors relied upon by independent modules.
Compounded Failure	psf__requests-2148	Mixed	Variable	Centralized introduced a regression while decentralized failed to meet the target specification entirely.

5.2.4 Qualitative Analysis of Unevaluated Tasks

Six task pairs could not be included in the quantitative comparison because a valid SWE-bench result was unavailable for at least one architecture, as shown in Table 5.3. Their generated patches were compared qualitatively with the reference patches included in the benchmark records. This comparison considers the implementation location, underlying correction, and behavioral scope. It does not establish that a generated patch is correct or classify the task as resolved.

For matplotlib__matplotlib-22711, the reference patch updates RangeSlider handle positions and corrects horizontal and vertical handle interaction. Both generated patches recognized the four-vertex polygon edge case and updated the slider handles when its value changed. The centralized patch explicitly closed a four-vertex polygon and updated both handle coordinates. The decentralized patch avoided accessing a nonexistent fifth vertex and updated the coordinate corresponding to the slider orientation.

Neither patch implemented all of the reference changes related to selecting and dragging vertical handles. The centralized patch was more explicit in preserving polygon and handle state, while the decentralized patch was smaller but suppressed possible exceptions during handle updates. Both patches addressed an important part of the issue but were narrower than the reference solution.

For scikit-learn__scikit-learn-10508, both generated patches closely matched the reference patch. All implementations validate input through column_or_1d and return an empty NumPy array for empty input to transform and inverse_transform. The reference patch checks the sample count through _num_samples, while the generated patches inspect the normalized array's size. This difference is minor for the benchmark inputs, and both generated patches passed the target tests during internal QA.

For psf__requests-2317, the two systems produced identical patches. Both replace builtin_str(method) with to_native_string(method), which is the central

behavioral change in the reference patch. The reference patch also removes the obsolete builtin_str import, whereas the generated patches leave it in place. This represents a minor cleanliness issue rather than a significant behavioral difference.

Table 5.3: Qualitative comparison and scoping analysis of un-graded generated patches against benchmark reference solutions, indexed by task identifier, reference strategy, agent alignment, and deviation characteristics.

Task Identifier	**Reference Patch Strategy**	**Agent Implementation Alignment**	**Deviation from Reference Scope**
matplotlib__matplotlib-22711	Explicit multi-directional handle tracking.	Resolved vertex constraints but omitted comprehensive drag states.	Narrower: Addressed immediate structural defect only.
scikit-learn__scikit-learn-10508	Input validation via _num_samples.	Validated dimensions using explicit array size properties.	Equivalently Scoped: Alternative syntax with identical execution logic.
psf__requests-2317	Type conversion to native string; removed dead import.	Implemented correct type conversion logic.	Minor Omission: Functional parity achieved, but left an obsolete import statement.
pydata__xarray-4493	Introduced deprecation warning; left conversion to callers.	Enforced automated attribute unwrapping at internal conversion steps.	Broader: Risked unexpected side-effects in unmapped call paths.
pylint-dev__pylint-7114	Modified core namespace package differentiation logic.	Centralized restructured module expansion; Decentralized targeted local path validation.	Variable: Centralized was overly broad; Decentralized matched local intent closely.
pylint-dev__pylint-7228	Caught regex compilation errors at the argument parsing layer.	Intercepted compilation errors; Decentralized also altered legacy evaluation paths.	Strong Alignment: Both captured core intent; Decentralized introduced redundant coverage.

For pydata__xarray-4493, both systems identified the ambiguity created when a DataArray is supplied within a variable-construction tuple. The reference patch introduces a deprecation warning but leaves conversion decisions to the callers, which are updated separately to pass the underlying data.

The centralized patch automatically replaces the DataArray with its .data attribute inside as_variable and at another conversion point. The decentralized patch is closer to the reference within as_variable because it adds the warning without rewriting the tuple, but it also changes the general as_compatible_data function to unwrap DataArray objects. Both generated solutions therefore introduce broader automatic conversion behavior than the reference patch. This could preserve lazy evaluation but may also affect unrelated call paths.

For pylint-dev__pylint-7114, the reference patch changes how Pylint distinguishes regular packages from directories without an __init__.py file. The centralized solution substantially restructures module expansion, including namespace handling, path normalization, file enumeration, and result deduplication. Although these changes are related to the issue, they have a much broader scope than the reference patch.

The decentralized patch handles the reported edge case more directly by checking whether a directory without an init.py file contains a Python module with the same name. It then uses that module as the resolved path. This is not identical to the reference implementation, but it is more narrowly connected to the reported defect than the centralized restructuring.

For pylint-dev__pylint-7228, both systems follow the central strategy of the reference patch. They catch regular-expression compilation errors and convert them into argparse errors containing the invalid pattern, location, and explanatory message. Both implementations also apply this behavior to comma-separated regular-expression options.

The centralized patch limits most of its changes to the argument transformation layer. The decentralized patch additionally modifies an older option-validation path. This may provide more consistent error handling, but it extends beyond the scope of the reference patch. Both generated patches remain strongly aligned with the intended solution.

Overall, the qualitative comparison shows that the systems generally identified the correct implementation areas even when external grading was unavailable. The LabelEncoder, Requests, and regular-expression patches were particularly close to their reference solutions. Greater uncertainty remained for RangeSlider, xarray, and module discovery, where the generated patches either implemented only part of the reference behavior or introduced broader changes.

Across both the valid and unevaluated cases, patch scope, completeness, and regression validation were the strongest indicators of solution quality. Communication topology influenced how the agents coordinated and selected implementation strategies, but the technical precision of the final patch had a more direct relationship with benchmark success.

5.2.5. Comparison with the Baseline

5.2.5.1 Scope and Limitations of the Baseline Comparison

The comparison, classified in Table 5.4 and Figure 5.1, with the single-agent baseline provides additional context for interpreting the task-level behavior of the two multi-agent architectures. However, this comparison is treated as a qualitative supplement rather than a complete system-level benchmark, because baseline outcomes were available for only a subset of the evaluated tasks. Therefore, the purpose of this subsection is not to make a broad statistical claim about single-agent and multi-agent systems, but to identify the types of tasks in which the MAS architectures demonstrated either comparable performance or clear architectural advantages.

Table 5.4: Task-level performance comparison between the gpt-5-mini single-agent baseline and the two multi-agent architectures across available subsets, categorized by comparative performance types.

Task Identifier	**Baseline (Single Agent)**	**Centralized**	**Decentralized**	**Performance Category**
astropy__astropy-14995 django__django-10914 pydata__xarray-4094 scikit-learn__scikit-learn-10297	**Resolved**	**Resolved**	**Resolved**	Parity (Low-complexity/Localized tasks)
astropy__astropy-14182 astropy__astropy-14365 sphinx-doc__sphinx-11445	Unresolved	**Resolved**	**Resolved**	Multi-Agent Architecture Advantage
matplotlib__matplotlib-23299	Unresolved	**Resolved**	Unresolved	Centralized Topology Advantage

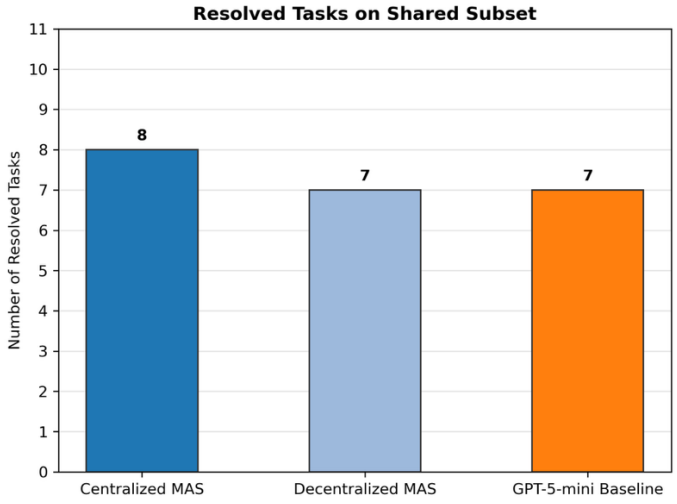


Figure 5.1. Comparative evaluation of Centralized MAS, Decentralized MAS, and the GPT-5-mini baseline by number of resolved tasks. All systems are evaluated on the same 11 task instances shared between the selected SWE-bench Lite evaluation set and SWE-bench Verified.

The available comparison shows three main categories of behavior. First, some tasks were resolved by the baseline, centralized MAS, and decentralized MAS, indicating performance parity across system designs. Second, some tasks were unresolved by the baseline but resolved by both MAS variants, suggesting an advantage from multi-agent decomposition and review. Third, one task was resolved only by the centralized MAS, indicating that the communication topology itself can influence repair quality when precise coordination and scope control are required.

5.2.5.2 Parity on Localized or Lower-Complexity Tasks

The first group consists of tasks resolved by all three systems: astropy__astropy-14995, django__django-10914, pydata__xarray-4094, and scikit-learn__scikit-learn-10297. These cases represent parity between the single-agent baseline and both MAS architectures. Their shared success suggests that, when the required repair is relatively localized and the expected behavioral change is clearly expressed by the failing tests, additional agent coordination does not necessarily create a measurable advantage.

This does not mean that the MAS workflow was ineffective in these cases. Rather, it indicates that the tasks did not strongly require the additional structure provided by multi-agent collaboration. A single agent was already able to identify the relevant code region and produce a correct patch, while the MAS systems reached the same outcome through a more distributed process. In such cases, the benefit of MAS is less visible in final correctness, because the task complexity is not high enough to expose meaningful differences between architectures.

This category is important because it establishes that the MAS architectures did not generally reduce patch quality on straightforward tasks. Both centralized and decentralized systems were able to match the baseline when the problem was sufficiently constrained. However, these cases also suggest that MAS may introduce additional coordination overhead without producing a corresponding correctness gain

when the repair does not require multi-step reasoning, broad context tracking, or careful trade-off management.

5.2.5.3 Multi-Agent Architecture Advantage

The second group consists of tasks unresolved by the baseline but resolved by both MAS architectures: astropy__astropy-14182, astropy__astropy-14365, and sphinx-doc__sphinx-11445. These cases provide the strongest evidence that the multi-agent structure can improve repair outcomes under certain task conditions.

A common feature of these tasks is that the required fix involved preserving a broader behavioral contract rather than changing only one isolated line of code. For astropy__astropy-14182, the systems needed to modify RestructuredText table output while maintaining compatibility with existing writer behavior. For astropy__astropy-14365, the repair required QDP command parsing to become case-insensitive without disrupting the surrounding parsing logic. For sphinx-doc__sphinx-11445, the systems needed to handle rst_prolog insertion while preserving correct parsing of section titles containing roles.

These tasks benefited from the MAS workflow because they required the system to connect the reported issue with surrounding implementation constraints. The multi-agent architectures provided additional opportunities for decomposition, inspection, and verification. Even though the centralized and decentralized systems used different communication structures, both were able to maintain enough task context to produce valid repairs. This suggests that, for moderately complex tasks involving format handling, parser behavior, or documentation preprocessing, multi-agent collaboration can provide a practical advantage over a single-agent repair process.

5.2.5.4 Centralized Topology Advantage

The third category is represented by matplotlib__matplotlib-23299, which was resolved by the centralized MAS but not by the baseline or the decentralized MAS.

This task is particularly useful because it isolates the effect of communication topology within the multi-agent setting. The result suggests that the centralized architecture's coordinator-based structure can improve repair precision when the task requires careful control over patch scope.

The issue required preserving the backend selected inside rc_context while still restoring the remaining runtime configuration. This is a narrow state-preservation requirement: the correct behavior was not to preserve all configuration changes, but to preserve one specific backend-related state while maintaining the broader semantics of rc_context. The centralized MAS produced a focused patch that reflected this distinction. By contrast, the decentralized MAS introduced broader configuration-preservation behavior, which increased the likelihood of unintended side effects.

This case supports the interpretation that centralized coordination can be valuable when a task requires explicit requirement tracking and restraint. The manager role appears to help maintain alignment between the problem statement, the implementation plan, and the final patch. Rather than allowing local agent decisions to accumulate into a broader behavioral change, the centralized workflow can constrain the repair around the specific requirement. This makes the centralized topology especially suitable for tasks where the main risk is not failing to find the relevant code, but modifying too much of the surrounding behavior.

5.2.5.5 Interpretation of Baseline-Level Differences

Overall, the comparison suggests that MAS provides the clearest benefit when the task requires more than localized code modification. In low-complexity or highly localized repairs, the single-agent baseline and both MAS architectures reached the same outcome. In these cases, the final correctness result shows parity rather than architectural separation. However, when the task involved broader behavioral preservation, parser logic, formatting rules, or state-management constraints, the MAS architectures demonstrated stronger performance.

The results also show that the advantage of MAS is not uniform across all multi-agent designs. While both MAS variants resolved several tasks that the baseline did not, the centralized topology showed an additional advantage in matplotlib__matplotlib-23299. This indicates that multi-agent collaboration alone is not the only factor influencing repair quality; the structure of communication between agents also matters. A centralized topology can provide stronger oversight and patch-scope control, while a decentralized topology may be more vulnerable to broader interpretation of the required change.

Thus, the baseline comparison reinforces the broader task-level findings of this study. Multi-agent systems are most valuable when software repair requires structured reasoning, sustained context, and explicit validation of behavioral constraints. The centralized MAS, in particular, appears better suited to tasks where correctness depends on preserving a narrow semantic distinction. At the same time, parity on simpler tasks indicates that MAS should be viewed as a targeted architectural improvement rather than a universal replacement for single-agent repair.

5.3 Efficiency and Cost Considerations

Correctness is the primary goal in SWE-bench evaluation, but efficiency is also important for practical deployment. Multi-agent systems can consume more resources than single-agent approaches because they involve multiple roles, repeated communication, and additional validation stages. For this reason, runtime and token usage are useful secondary metrics.

The decentralized MAS had a lower average execution duration than the centralized MAS. This may be because decentralized handoffs allow agents to move directly to the next relevant specialist without always returning through the Project Manager. In practice, this could make decentralized coordination attractive for workflows where response time is important.

However, the decentralized MAS also consumed more tokens on average. This indicates that faster execution did not translate into lower model cost. One possible

explanation is that decentralized agents may duplicate context across handoffs or spend more tokens clarifying task state. The centralized MAS, by contrast, may benefit from a more compact coordination pattern because the Project Manager controls the task flow and can reduce redundant exchanges.

These results show that efficiency should be evaluated along more than one dimension. A system may be faster in wall-clock time but more expensive in token usage. For real-world use, the preferred architecture may depend on whether the priority is task success, latency, cost, or predictability.

5.4 Interpreting the Token-Latency Inversion

The aggregate results present an apparent inversion: centralized execution used fewer tokens but required more wall-clock time, whereas decentralized execution used more tokens but completed sooner. This pattern is not contradictory because token consumption and elapsed time measure different aspects of execution. Total tokens are an additive measure of all model context processed across the run. Wall-clock duration instead reflects the execution's critical path, including model-request overhead, sequential handoffs, tool calls, test execution, Docker operations, and waiting time. Consequently, fewer processed tokens do not necessarily imply a shorter run.

The centralized topology provides one plausible explanation for the lower token total. Because specialist work is routed through the Project Manager, the manager can frame a narrower task for the next role, consolidate intermediate results, and maintain a single coordination state through the project board. This mediation can reduce repeated technical discussion and unnecessary propagation of the full working context. In this sense, the manager operates as a context filter: centralized coordination may perform more control transitions while transmitting less aggregate language-model context.

The same mechanism can increase elapsed time. A specialist must return control to the Project Manager before the next stage is selected, and the manager must

interpret the result and issue another handoff. These operations lie on a serial coordination path. Even when a manager message contains relatively few tokens, it introduces another model request, scheduling delay, and handoff boundary before technical work continues. The observed centralized result is therefore consistent with lower context volume but greater orchestration latency.

The decentralized topology exhibits the complementary trade-off. Permitted specialist-to-specialist handoffs shorten selected coordination paths by allowing review or validation work to continue without first returning through the manager. This can reduce fixed handoff and decision latency. However, technical agents may receive and reprocess more detailed implementation context, test evidence, and prior conversation history. Local correction loops can also repeat this context across multiple model calls. Decentralized routing can therefore make progress sooner while accumulating a larger total token volume.

This interpretation does not require true parallel execution. The evaluated workflow remains largely sequential; the potential latency benefit comes from removing intermediate routing stages from the critical path rather than executing every specialist simultaneously. Moreover, token count is not a complete runtime model. Two runs with identical token totals may differ because of the number and size of API calls, provider response time, cache behavior, MCP operations, repository size, or test-suite duration.

Call-level timing evidence suggests that model-provider response speed was not a systematic source of difference between the two arms. Approximate LLM response intervals were reconstructed from timestamps in the outer AutoGen event logs and the inner Engineer trajectories, then averaged within each run before paired comparison. The mean per-run interval was 7.74 seconds for centralized execution and 7.61 seconds for decentralized execution. The paired centralized-minus-decentralized difference was 0.13 seconds, with a 95% bootstrap interval from -0.71 to 1.00 seconds and no detectable paired shift under a Wilcoxon test ($p = 0.826$). This

finding does not imply that LLM inference time is absent from wall-clock duration. Rather, it indicates that comparable per-call response latency was shared by both experimental arms and is therefore unlikely to explain the observed difference in their mean run duration. The architectures may still differ in how many calls they place and where those calls occur on the critical path.

The distributional evidence requires a cautious conclusion. Its higher mean token use was substantially influenced by expensive tail cases. The paired confidence intervals also crossed zero for both token and duration differences. Therefore, the results support a descriptive mechanism rather than a causal claim: centralized routing appears to favor context economy at the cost of additional serial coordination, while decentralized routing appears to favor a shorter coordination path at the risk of context duplication and costly local loops. Confirming this explanation would require additional measurements of per-role token volume, number of handoffs, model-call count, tool time, and waiting time.

5.5 Implications for Multi-Agent Software Engineering

The findings of this study carry significant implications for the architectural design of future multi-agent systems tailored for automated software engineering. Beyond the surface-level metrics of task resolution, the experimental data suggests that communication topology should not be treated as a static backdrop, but rather as a primary, tunable lever that directly dictates system reliability, latency, and operational cost.

First, the competitive performance of both architectures implies that there is no universally superior communication topology for software development tasks. Instead, system designers must transition from asking which architecture is better to evaluating when a specific structure should be deployed. The slight performance edge of the centralized MAS underscores that when a task demands strict adherence to software engineering principles—such as precise scope control and multi-requirement tracking—a hub-and-spoke model is necessary to prevent contextual

drift. Conversely, for rapid prototyping, localized refactoring, or workflows where execution latency is the primary bottleneck, the direct handoffs of a decentralized swarm offer distinct operational advantages.

Second, the divergence between execution duration and token consumption highlights a multi-dimensional optimization problem for practical deployments. The fact that the decentralized system was faster in wall-clock time yet more expensive in token usage challenges the assumption that reducing routing friction inherently lowers operational overhead. In production environments, software engineering agents will operate under real-world constraints like API rate limits, model financial costs, and user patience. Designers must carefully weigh whether their operational constraints favor the predictable, compact token usage of a central manager or the rapid, parallelizable velocity of a decentralized network.

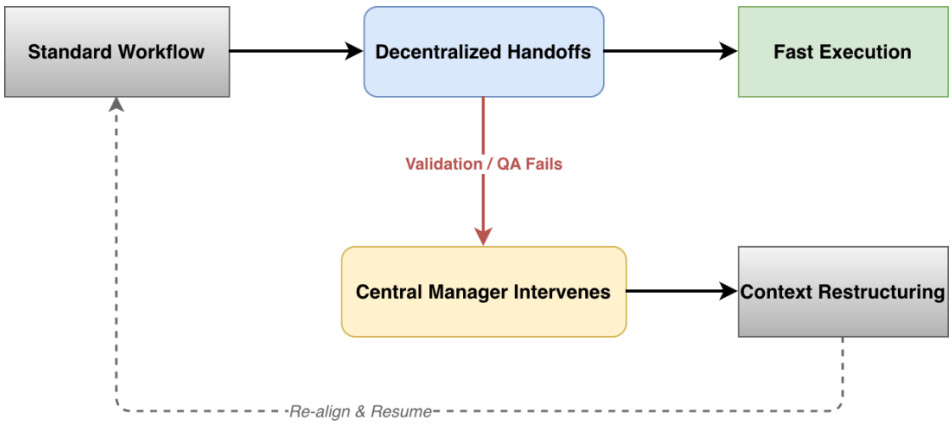


Figure 5.2. Flowchart of the proposed dynamically hybrid coordination model, illustrating standard decentralized execution pathways transitioning to a centralized managerial fallback configuration upon validation or quality assurance failures.

Finally, the localized failures observed in both systems point toward a clear evolutionary path for multi-agent software engineering: the adoption of dynamically hybrid coordination models, as proposed in Figure 5.2. Rather than enforcing a

dogmatic choice between rigid centralization and autonomous swarms, future frameworks should implement an architecture that adapts to the lifecycle of the task.

In such a hybrid paradigm, the system leverages decentralized handoffs as its default state to maximize speed and allow specialist agents (e.g., the Engineer and Reviewer) to collaborate with minimal latency. However, if a validation check fails, an agent disagreement occurs, or the task context grows excessively fragmented, the system triggers a structural fallback: a centralized coordinator steps in to audit the state, re-align the agents, and enforce a corrected path forward. By treating communication topology as a dynamic, runtime variable, future multi-agent systems can successfully combine the rigorous quality control of centralized oversight with the agile execution of decentralized design.

5.6 Architecture Decision Framework

Based on the experimental findings, the selection of a multi-agent communication topology should be treated as a context-dependent architectural decision rather than a fixed design preference. The results show that centralized and decentralized MAS structures optimize for different operational goals. The centralized system achieved higher correctness and lower token consumption, while the decentralized system achieved lower execution time but consumed more tokens and showed greater vulnerability to context drift. Therefore, the appropriate topology depends on the constraints of the software engineering task and the deployment environment.

The proposed decision framework in Table 5.5 evaluates MAS topology selection across four main dimensions: task complexity, correctness requirements, latency constraints, and cost sensitivity. These dimensions reflect the main trade-offs observed in the experiment and provide practical guidance for selecting centralized, decentralized, or hybrid coordination.

For tasks with high complexity, ambiguous requirements, or high regression risk, a centralized topology is more appropriate. In these cases, the Project Manager

serves as a coordination and control layer that maintains task intent, routes work through a predictable sequence, and prevents premature completion. This is especially useful for multi-file repairs, tasks involving multiple interacting requirements, or changes where a narrow implementation scope must be preserved. Although this structure introduces additional routing time, the experiment suggests that this cost is justified when correctness and regression control are the primary concerns.

Table 5.5. Architecture decision matrix for MAS topology selection, summarizing recommended centralized, decentralized, and hybrid coordination strategies across task complexity, correctness risk, latency, cost sensitivity, ambiguity, and fault tolerance.

Decision Factor	Centralized	Decentralized	Hybrid
Task complexity	Recommended for high-complexity, multi-file, or multi-requirement tasks.	Recommended for localized tasks with a clear implementation target.	Recommended when task complexity is uncertain or may increase during execution.
Correctness and regression risk	Recommended when correctness is critical and regression tolerance is low.	Suitable only when the change is low-risk and regression impact is limited.	Recommended when validation failures should trigger escalation to centralized oversight.
Execution latency	Suitable when slower execution is acceptable in exchange for stronger coordination.	Recommended when fast turnaround is the primary operational goal.	Recommended when speed is desired but fallback protection is still needed.
Token and cost sensitivity	Recommended when predictable token usage and cost control are important.	Suitable when wall-clock speed is more important than token cost.	Recommended when cost should be controlled dynamically through escalation rules.
Requirement ambiguity	Recommended when requirements require interpretation, prioritization, or reconciliation.	Suitable when requirements are explicit, narrow, and directly testable.	Recommended when ambiguity may emerge during implementation or validation.

Fault tolerance	Recommended when failures must be detected and corrected through a clear coordinating authority.	Suitable when independent specialist autonomy is preferred and failure impact is low.	Recommended when failures should trigger managerial intervention and workflow realignment.

For tasks with low complexity, clear repair targets, and strong latency constraints, a decentralized topology may be preferable. Direct handoffs between specialist agents reduce managerial routing overhead and allow the workflow to progress more quickly. This makes decentralized coordination suitable for localized bug fixes, simple refactoring, or rapid prototyping tasks where the implementation path is already clear. However, this topology should be used carefully when regression risk is high, because the absence of a central coordinator can make context drift and incomplete requirement tracking more likely.

For mixed or uncertain tasks, a hybrid topology is the most suitable design direction. A hybrid MAS can begin with decentralized handoffs to preserve speed during routine implementation and review, while dynamically activating centralized oversight when risk increases. Triggers for this fallback may include failed QA validation, repeated agent disagreement, excessive token growth, ambiguous requirements, or detection of a regression-producing patch. This approach allows the system to combine the execution velocity of decentralized coordination with the corrective control of centralized management.

This framework shows that centralized and decentralized MAS architectures should not be viewed as competing universal solutions. Instead, they represent different points in a design space. Centralized coordination is best suited for reliability, traceability, and regression-sensitive work. Decentralized coordination is best suited for speed, autonomy, and low-risk localized changes. Hybrid coordination offers the most promising direction for future systems because it treats topology as a runtime variable rather than a static architectural choice.

In practical software engineering deployments, this means that MAS designers should select topology according to the dominant risk of the task. If the main risk is incorrect or over-scoped implementation, centralized coordination should be preferred. If the main risk is slow response time, decentralized coordination may be sufficient. If both risks are present, the system should begin with decentralized execution but maintain a centralized fallback mechanism for validation failure, context fragmentation, or unresolved disagreement. This decision framework therefore translates the experimental findings into actionable guidance for designing future multi-agent software engineering systems.

CHAPTER 6. CONCLUSION

This paper investigated the use of multi-agent systems for automated software engineering tasks, with a particular focus on how agent communication structure affects task-solving performance. Two multi-agent architectures were implemented and evaluated: a centralized system based on a hub-and-spoke coordination pattern, and a decentralized system based on direct handoffs between specialist agents. Both systems were evaluated on a fixed subset of SWE-bench Lite tasks using the same experimental pipeline and grading procedure.

6.1. Summary of Contributions

The first contribution of this paper is the design and implementation of a centralized multi-agent software engineering system. This system organizes the workflow around a Project Manager agent, which coordinates specialist agents responsible for architecture, implementation, code review, and quality assurance. The centralized design provides a controlled workflow in which task progress is routed through a central coordinator.

The second contribution is the design and implementation of a decentralized multi-agent software engineering system. In this architecture, agents can hand off work directly to other agents according to the current state of the task. This design reduces dependence on a single coordinating agent and allows more flexible communication between specialists.

The third contribution is the construction of an evaluation pipeline for comparing multi-agent software engineering systems on SWE-bench Lite. The pipeline prepares benchmark instances, executes each system under controlled conditions, collects generated patches and telemetry, and submits the patches to the SWE-bench grading harness.

The final contribution is an empirical comparison between the centralized and decentralized architectures. The experiment provides evidence about the trade-offs

between reliability, runtime, and token usage in different multi-agent communication structures.

6.2 Answers to Research Objectives

This section revisits the two research objectives introduced at the beginning of the study and summarizes how the experimental results answer them. Rather than repeating the full analysis from the previous chapter, the discussion here synthesizes the main empirical findings into direct answers about coordination cost, runtime efficiency, patch precision, and architectural trade-offs. The goal is to connect the benchmark results back to the broader motivation of the paper: understanding how communication topology shapes the behavior and effectiveness of multi-agent software engineering systems.

6.2.1 Objective 1: Quantifying the Manager Tax and Operational Efficiency

The first objective of this paper was to evaluate how communication topology affects coordination cost and runtime efficiency in multi-agent software engineering systems. The results show that decentralization reduced execution time, but did not reduce overall computational cost. The decentralized MAS completed tasks in an average of 443.54 seconds, compared with 584.36 seconds for the centralized MAS. This suggests that removing the Project Manager as a required routing point can reduce some coordination delay, especially in iterative loops between implementation, review, and testing.

However, the runtime advantage of the decentralized architecture was relatively modest and came with a substantial increase in token consumption. The decentralized MAS consumed an average of 551,079 tokens per task, compared with 433,991 tokens for the centralized MAS. This represents approximately 27% higher token usage. The result suggests that direct agent-to-agent handoffs can reduce wall-clock latency, but may require more repeated context reconstruction, clarification, and state realignment across agents.

Therefore, the “Manager Tax” in the centralized MAS appears primarily as additional execution time caused by routing work through a central coordinator. In contrast, the “Decentralized Tax” appears primarily as token overhead caused by distributed coordination. The centralized system was slower but more predictable in token usage, while the decentralized system was faster but more expensive to run.

6.2.2 Objective 2: Evaluating Patch Precision and Code Correctness

The second objective was to evaluate whether hierarchical coordination improves patch precision and functional correctness. Based on the SWE-bench Lite evaluation, the centralized MAS achieved the higher resolved rate, solving 35 out of 50 benchmark instances, or 70.00%. The decentralized MAS solved 33 out of 50 instances, corresponding to a resolved rate of 66.00%.

The difference in aggregate correctness was modest, but the task-level analysis provides a clearer explanation. Among valid paired comparisons, both architectures produced the same outcome on 90.9% of tasks. This indicates that communication topology did not radically change performance across most benchmark instances. However, in the cases where the systems diverged, the centralized MAS was more successful when the task required careful requirement tracking, controlled patch scope, or regression prevention.

The centralized architecture benefited from a more structured hub-and-spoke workflow. The Project Manager helped preserve task intent across the design, implementation, review, and QA stages. This appeared useful in tasks where a patch needed to satisfy multiple related requirements or avoid broad behavioral changes. By contrast, the decentralized MAS performed well when the task could be solved through a direct localized modification, but it was more vulnerable to incomplete requirement tracking and regression-producing patches.

Thus, the centralized MAS provided a small but meaningful correctness advantage in this experiment. Its main strength was not that it solved every task better,

but that it more often constrained the workflow toward precise and regression-safe patches.

6.2.3 Synthesis: Architectural Trade-offs and System Selection

Taken together, the findings show that neither communication topology is universally superior. The centralized MAS and decentralized MAS present different operational trade-offs. The centralized system favors correctness, patch precision, and token predictability, but pays for these advantages through slightly longer execution time. The decentralized system favors faster execution and more autonomous specialist interaction, but pays through higher token usage and greater risk of context drift across handoffs.

For tasks that require broad requirement tracking, careful coordination, or regression-sensitive changes, the centralized architecture appears more suitable. Its explicit coordination structure helps preserve a stable view of the task and reduces the likelihood that agents optimize only for the immediate failing test. For narrower tasks with a clearly localized implementation point, the decentralized architecture may be preferable because direct handoffs can reduce routing overhead and allow specialists to act more quickly.

The results therefore support a trade-off-based view of multi-agent system design. A central manager improves control and reliability, while decentralized handoffs improve speed and flexibility. Future systems may benefit from combining both approaches: allowing direct specialist handoffs during routine progress, while reintroducing centralized oversight when the task becomes ambiguous, when QA detects regressions, or when multiple agents need to reconcile conflicting assumptions.

6.3 Limitations

This study has several limitations that should be considered when interpreting the results. First, the experiment was conducted on a subset of 50 SWE-bench Lite

instances. Although the selected subset includes tasks from multiple repositories and covers a range of software engineering problems, it does not fully represent the diversity of real-world maintenance tasks. Larger benchmark coverage would be required to determine whether the observed patterns generalize across a wider range of repositories, programming styles, and defect types.

Second, each benchmark instance was executed once per system. This limits the ability to measure run-to-run variance. Because LLM-based systems are probabilistic, repeated executions of the same task may produce different repair strategies, different patches, or different failure modes. As a result, the reported resolution rates should be interpreted as outcomes of the specific experimental runs rather than as definitive estimates of each architecture's stable long-term performance.

Third, the single-agent baseline comparison was incomplete. Baseline grading results were available for only part of the benchmark subset, so the primary empirical comparison in this study focused on the centralized and decentralized multi-agent architectures. The available baseline results were still useful for qualitative task-level interpretation, but they were not sufficient to support a full quantitative comparison between single-agent and multi-agent repair systems.

Fourth, the results may be influenced by implementation-specific design choices. These include prompt design, role definitions, task handoff rules, timeout limits, context-sharing mechanisms, and QA procedures. Different implementations of centralized or decentralized coordination could produce different results. Therefore, the findings should be understood as evidence about the systems implemented in this study, rather than as universal conclusions about all possible centralized or decentralized multi-agent software engineering architectures.

Finally, several task outcomes were affected by benchmark execution and evaluation-environment issues. These cases were excluded from the main paired task-level comparison, but they still limit the completeness of the evaluation. Although

qualitative patch comparisons were used where possible, environment-related failures make it difficult to determine whether those patches would have passed under a fully successful SWE-bench execution.

6.4 Future Work

Future work could extend this study in several directions. First, the experiment should be repeated on a larger benchmark set, including the full SWE-bench Lite dataset and, where feasible, the official SWE-bench benchmark or other software engineering benchmarks. A larger evaluation would provide stronger evidence about the generality of the results and make it possible to analyze performance across a broader range of repositories, issue types, and repair complexities.

Second, future evaluations should include multiple runs per task. Repeated trials would allow measurement of variance across executions and provide a clearer understanding of the stability of each architecture. This is especially important for LLM-based systems, where differences in sampling, context construction, and intermediate reasoning can lead to different patches for the same benchmark instance.

Third, future work should include a complete and consistently evaluated single-agent baseline. This would make it possible to measure more directly whether the additional coordination cost of multi-agent systems is justified by improvements in correctness, robustness, or task coverage. A complete baseline would also help distinguish improvements caused by multi-agent coordination from improvements caused by model capability or implementation-specific prompting.

Fourth, further analysis should examine the task characteristics that led to different outcomes between the centralized and decentralized architectures. In particular, future studies could classify tasks by repair locality, number of affected modules, dependency complexity, regression risk, and requirement ambiguity. Such analysis would provide more practical guidance for selecting an appropriate coordination topology for a given software engineering task.

Finally, future work should prototype and evaluate the dynamically hybrid coordination model proposed in the discussion. Such a system could default to decentralized handoffs for faster execution, while dynamically activating centralized oversight when validation failures, repeated agent disagreement, or high regression risk are detected. This would provide a concrete framework for balancing the latency advantages of decentralized coordination with the patch precision and scope control of centralized management.

6.5 Final Remarks

The findings of this study demonstrate that agent communication structure is an important design factor in automated software engineering systems. The difference between the centralized and decentralized architectures was meaningful, but not absolute. Both systems were capable of resolving complex real-world software repair tasks, yet they exhibited different operational strengths. The centralized architecture provided stronger correctness, lower token consumption, and more predictable coordination, while the decentralized architecture offered faster execution through more direct agent-to-agent interaction.

These results suggest that the choice of multi-agent topology should depend on the constraints of the deployment context. When patch precision, regression control, and cost predictability are the primary concerns, centralized coordination is likely to be more appropriate. When faster iteration and greater workflow autonomy are more important, decentralized coordination may be preferable, provided that stronger validation mechanisms are added to control regression risk.

In conclusion, this paper shows that multi-agent architectures can support automated bug fixing in real-world codebases, but their effectiveness depends heavily on how agents communicate, share context, and validate proposed changes. By identifying the trade-offs between centralized oversight and decentralized autonomy, this study provides an empirical foundation for future adaptive hybrid systems

thatcombine the reliability of managerial coordination with the speed and flexibility of specialist collaboration.

REFERENCES

[1] V. Nair et al., “The Context Window Problem: Scaling Agents Beyond Token Limits”, Factory.ai, Aug. 2025. [Online]. Available: https://factory.ai/news/context-window-problem

[2] JetBrains Research, “Cutting Through the Noise: Smarter Context Management for LLM-Powered Agents”, JetBrains Blog, Dec. 2025. [Online]. Available: https://blog.jetbrains.com/research/2025/12/efficient-context-management

[3] S. Hong et al., “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework”, in Proc. Int. Conf. Learn. Represent. (ICLR), 2024, doi: 10.48550/arXiv.2308.00352.

[4] C. Qian et al., “ChatDev: Communicative Agents for Software Development”, in Proc. Assoc. Comput. Linguist. (ACL), 2024, doi: 10.18653/v1/2024.acl-long.810.

[5] C. Yang et al., “AgentNet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems”, 2025, doi: 10.32388/WS0VIM.

[6] G. Kim et al., “Towards a Science of Scaling Agent Systems”, 2025,

doi: 10.48550/arXiv.2512.08296.

[7] L. Du et al., “Improving Factuality and Reasoning in Language Models through Multiagent Debate”, 2023, doi: 10.48550/arXiv.2305.14325.

[8] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”, in Adv. Neural Inf. Process. Syst. (NeurIPS), 2022, doi: 10.48550/arXiv.2201.11903.

[9] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning”, in Adv. Neural Inf. Process. Syst. (NeurIPS), 2023, doi: 10.48550/arXiv.2303.11366.

[10] G. Li et al., “CAMEL: Communicative Agents for ‘Mind’ Exploration of Large Scale Language Model Society”, in Adv. Neural Inf. Process. Syst. (NeurIPS), 2023, doi: 10.48550/arXiv.2303.17760.

[11] W. Chen et al., “AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors”, 2023, doi: 10.48550/arXiv.2308.10848.

[12] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications”, Microsoft Research, 2023, doi: 10.48550/arXiv.2308.08155.

[13] M. Sharma et al., “Towards Understanding Sycophancy in Language Models”, Anthropic Research, 2023, doi: 10.48550/arXiv.2310.13548.

[14] R. Cohen et al., “LM-vs-LM: The Dynamics of Inter-Model Disagreement”, 2023, doi: 10.48550/arXiv.2305.13281.

[15] D. Chen et al., “CodeR: Issue Resolving with Multi-Agent and Task Graphs”, 2024, doi: 10.48550/arXiv.2406.01304.

[16] Y. Zhang et al., “AutoCodeRover: Autonomous Program Improvement”, 2024, doi: 10.48550/arXiv.2404.05427.

[17] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., & Press, O, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. The Thirty-eighth Annual Conference on Neural Information Processing Systems”, 2024, doi: 10.48550/arXiv.2405.15793.